

# **CSE 208: Data Structure and Algorithms II**

## **Introduction and Graph Basics**

---

Dr. Mohammed Eunos Ali

Professor

CSE, BUET

Slides are from Dr Tanzima Hashem, [tanzimahashem@cse.buet.ac.bd](mailto:tanzimahashem@cse.buet.ac.bd)

# GRAPHS

? A graph  $G = (V, E)$

- $V$  = set of vertices
- $E$  = set of edges = subset of  $V \times V$
- Thus  $|E| = O(|V|^2)$



# GRAPH VARIATIONS

## ? Variations:

- A *connected graph* has a path from every vertex to every other
- In an *undirected graph*:
  - ? Edge  $(u,v) = \text{edge } (v,u)$
  - ? No self-loops
- In a *directed graph*:
  - ? Edge  $(u,v)$  goes from vertex  $u$  to vertex  $v$ , notated  $u \rightarrow v$
  - ? Self loops are allowed.



# GRAPH VARIATIONS

? More variations:

- A *weighted graph* associates weights with either the edges or the vertices
  - ? E.g., a road map: edges might be weighted w/ distance
- A *multigraph* allows multiple edges between the same vertices
  - ? E.g., the call graph in a program (a function can get called from multiple points in another function)



# GRAPHS

- ? We will typically express running times in terms of  $|E|$  and  $|V|$  (often dropping the  $|$ 's)
  - If  $|E| \approx |V|^2$  the graph is *dense*
  - If  $|E| \approx |V|$  the graph is *sparse*
- ? If you know you are dealing with dense or sparse graphs, different data structures may make sense



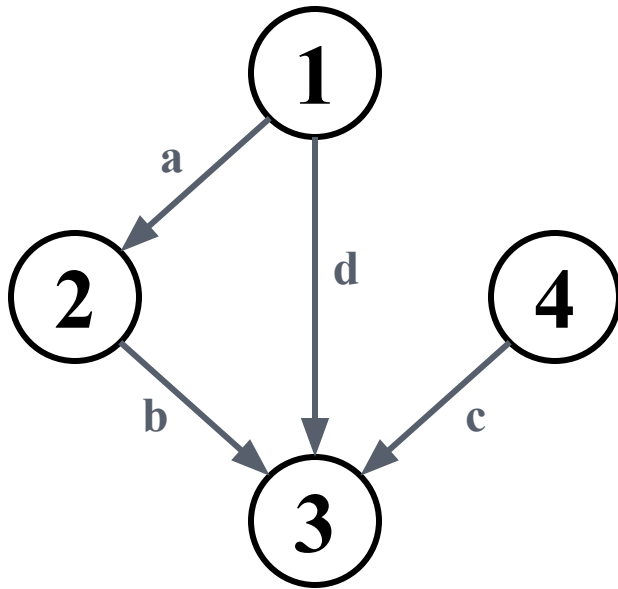
# REPRESENTING GRAPHS

- ? Assume  $V = \{1, 2, \dots, n\}$
- ? An *adjacency matrix* represents the graph as a  $n \times n$  matrix  $A$ :
  - $A[i, j] = 1$  if edge  $(i, j) \in E$  (or weight of edge)  
 $= 0$  if edge  $(i, j) \notin E$



# GRAPHS: ADJACENCY MATRIX

? Example:

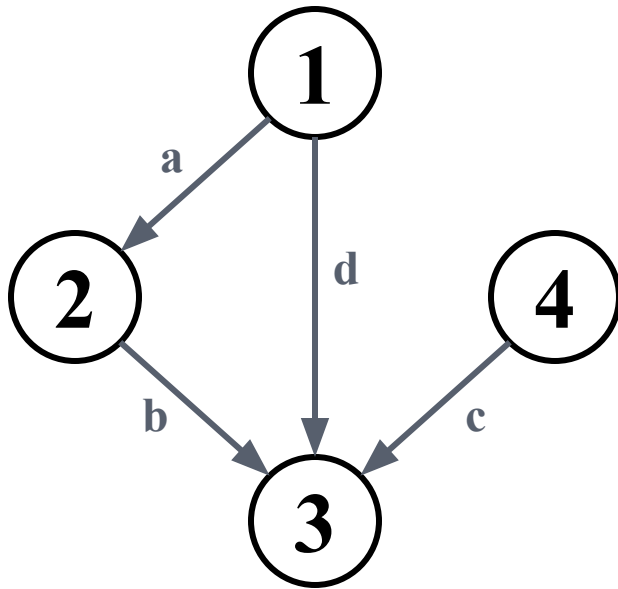


| A | 1 | 2 | 3  | 4 |
|---|---|---|----|---|
| 1 |   |   |    |   |
| 2 |   |   |    |   |
| 3 |   |   | ?? |   |
| 4 |   |   |    |   |



# GRAPHS: ADJACENCY MATRIX

? Example:



| A | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 |
| 2 | 0 | 0 | 1 | 0 |
| 3 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 1 | 0 |





# GRAPHS: ADJACENCY MATRIX

? **Space:**  $\Theta(V^2)$ .

- Not memory efficient for large graphs.

? **Time:** to list all vertices adjacent to  $u$ :  $\Theta(V)$ .

? **Time:** to determine if  $(u, v) \in E$ :  $\Theta(1)$ .



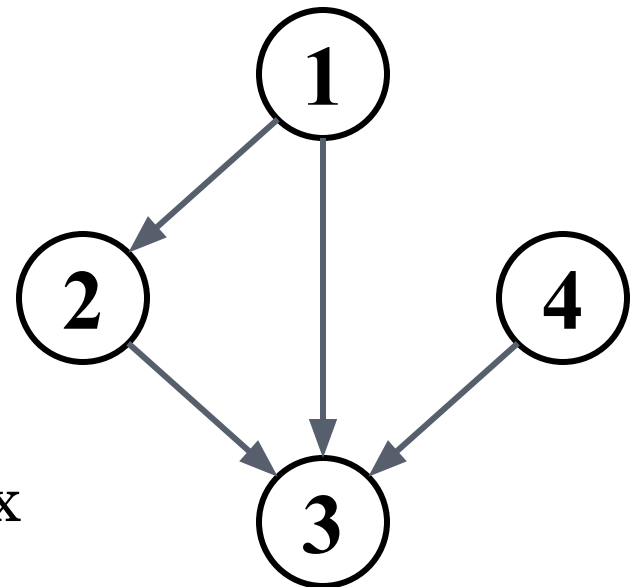
# GRAPHS: ADJACENCY MATRIX

- ? The adjacency matrix is a dense representation
  - Usually too much storage for large graphs
  - But can be very efficient for small graphs
- ? Most large interesting graphs are sparse
  - E.g., planar graphs, in which no edges cross, have  $|E| = O(|V|)$  by Euler's formula
  - For this reason the *adjacency list* is often a more appropriate representation



# GRAPHS: ADJACENCY LIST

- ? Adjacency list: for each vertex  $v \in V$ , store a list of vertices adjacent to  $v$
- ? Example:
  - $\text{Adj}[1] = \{2,3\}$
  - $\text{Adj}[2] = \{3\}$
  - $\text{Adj}[3] = \{\}$
  - $\text{Adj}[4] = \{3\}$
- ? Variation: can also keep a list of edges coming *into* vertex



# GRAPHS: ADJACENCY LIST

## ? For directed graphs:

- Sum of lengths of all adj. lists is

$$\sum_{v \in V} \text{out-degree}(v) = |E|$$

← No. of edges  
leaving  $v$

- Total storage:  $\Theta(V+E)$

## ? For undirected graphs:

- Sum of lengths of all adj. lists is

$$\sum_{v \in V} \text{degree}(v) = 2|E|$$

← No. of edges incident on  $v$ . Edge  $(u,v)$  is  
incident on vertices  $u$  and  $v$ .

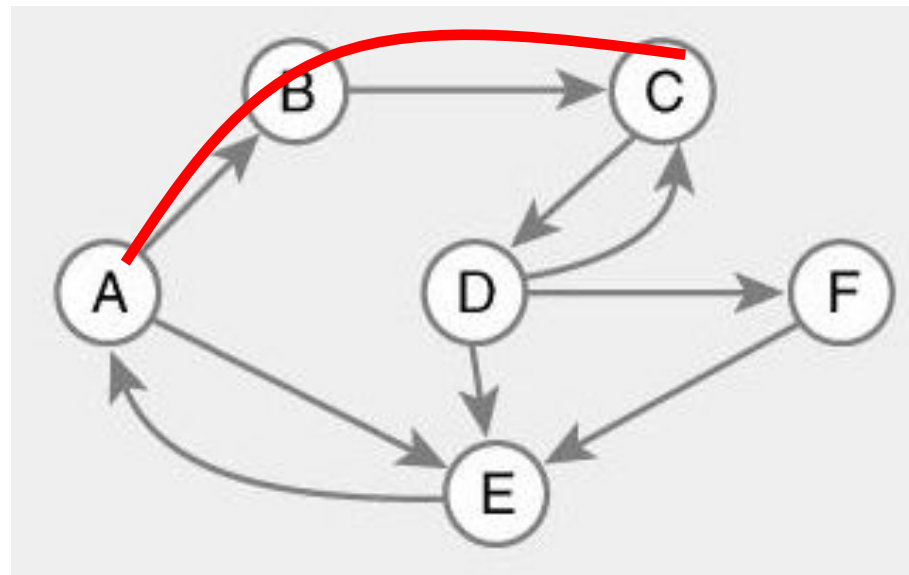
- Total storage:  $\Theta(V+E)$



# GRAPH DEFINITIONS

## ? Path

- Sequence of nodes  $n_1, n_2, \dots, n_k$
- Edge exists between each pair of nodes  $n_i, n_{i+1}$
- Example
  - ? A, B, C is a path



# GRAPH DEFINITIONS

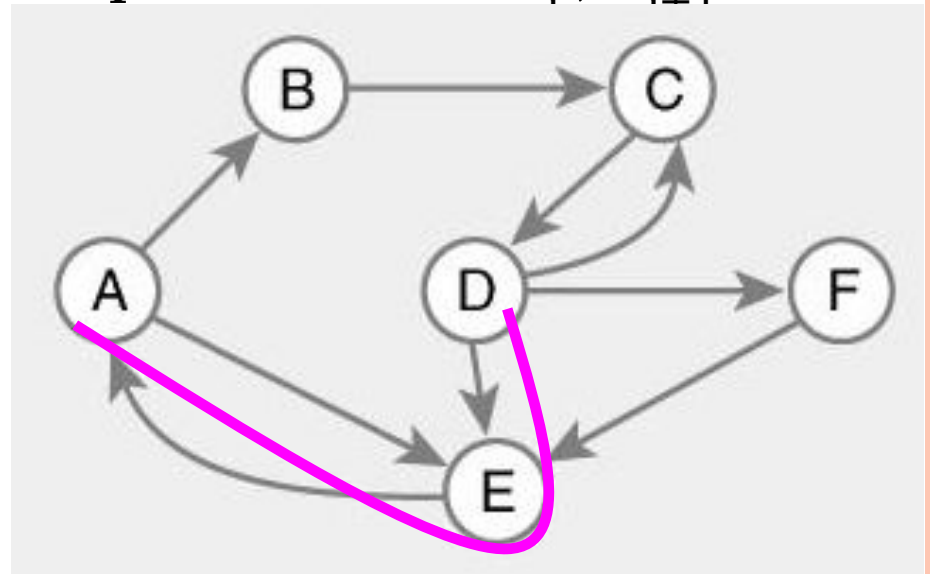
## ? Path

- Sequence of nodes  $n_1, n_2, \dots, n_k$
- Edge exists between each pair of nodes  $n_i, n_{i+1}$

- Example

? A, B, C is a path

? A, E, D is not a path

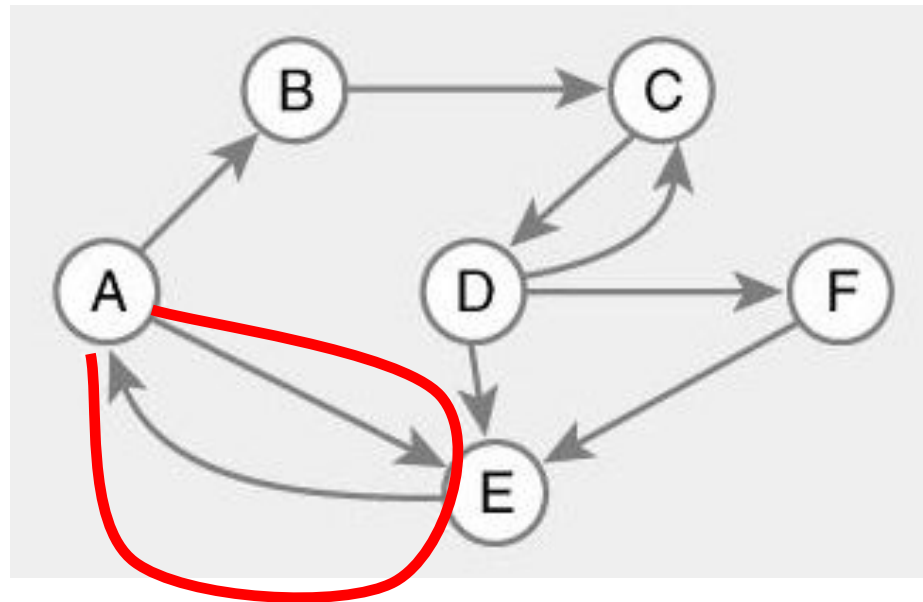


# GRAPH DEFINITIONS

## ? Cycle

- Path that ends back at starting node
- Example

? A, E, A



# GRAPH DEFINITIONS

## ? Cycle

- Path that ends back at starting node

- Example

? A, E, A

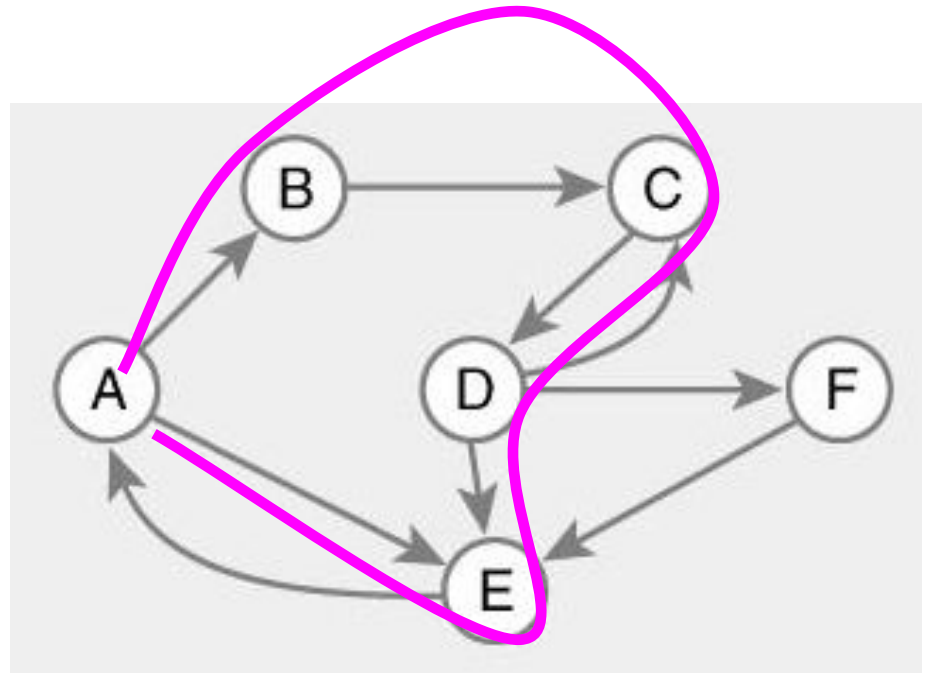
? A, B, C, D, E, A

## ? Simple path

- No cycles in path

## ? Acyclic graph

- No cycles in graph





# GRAPH SEARCHING

- ? Given: a graph  $G = (V, E)$ , directed or undirected
- ? Goal: methodically explore every vertex and every edge
- ? Ultimately: build a tree on the graph
  - Pick a vertex as the root
  - Choose certain edges to produce a tree
  - Note: might also build a *forest* if graph is not connected



# BREADTH-FIRST SEARCH

- ? “Explore” a graph, turning it into a tree
  - One vertex at a time
  - Expand frontier of explored vertices across the *breadth* of the frontier
- ? Builds a tree over the graph
  - Pick a *source vertex* to be the root
  - Find (“discover”) its children, then their children, etc.



# BFS - VERSION -1

BFS (s,Adj)

level = {s:0}

parent = {s:null}

i = 0

frontiers = [s]

while frontiers:

    next = []

    for u in frontiers

        for v in Adj[u]

            if v is not in level

                level[v] = i

                paren[v] = u

                next.append(v)

    i = i+1

    frontiers = next



# BREADTH-FIRST SEARCH

? **Input:** Graph  $G = (V, E)$ , either directed or undirected, and *source vertex*  $s \in V$ .

? **Output:**

- $d[v]$  = distance (smallest # of edges, or shortest path) from  $s$  to  $v$ , for all  $v \in V$ .  $d[v] = \infty$  if  $v$  is not reachable from  $s$ .
- $\pi[v] = u$  such that  $(u, v)$  is last edge on shortest path  $s \rightsquigarrow v$ .
  - ?  $u$  is  $v$ 's **predecessor**.
- Builds breadth-first tree with root  $s$  that contains all reachable vertices.



# BREADTH-FIRST SEARCH

- ? Associate vertex “colors” to guide the algorithm
  - White vertices have not been discovered
    - ? All vertices start out white
  - Grey vertices are discovered but not fully explored
    - ? They may be adjacent to white vertices
  - Black vertices are discovered and fully explored
    - ? They are adjacent only to black and gray vertices
- ? Explore vertices by scanning adjacency list of grey vertices



## BFS(G,s)

```
1. for each vertex  $u$  in  $V[G] - \{s\}$ 
2    $color[u] \leftarrow \text{white}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{nil}$ 
5  $color[s] \leftarrow \text{gray}$ 
6  $d[s] \leftarrow 0$ 
7  $\pi[s] \leftarrow \text{nil}$ 
8  $Q \leftarrow \Phi$ 
9  $\text{enqueue}(Q,s)$ 
10 while  $Q \neq \Phi$ 
11    $u \leftarrow \text{dequeue}(Q)$ 
12   for each  $v$  in  $\text{Adj}[u]$ 
13     if  $color[v] = \text{white}$ 
14        $color[v] \leftarrow \text{gray}$ 
15        $d[v] \leftarrow d[u] + 1$ 
16        $\pi[v] \leftarrow u$ 
17        $\text{enqueue}(Q,v)$ 
18  $color[u] \leftarrow \text{black}$ 
```

initialization

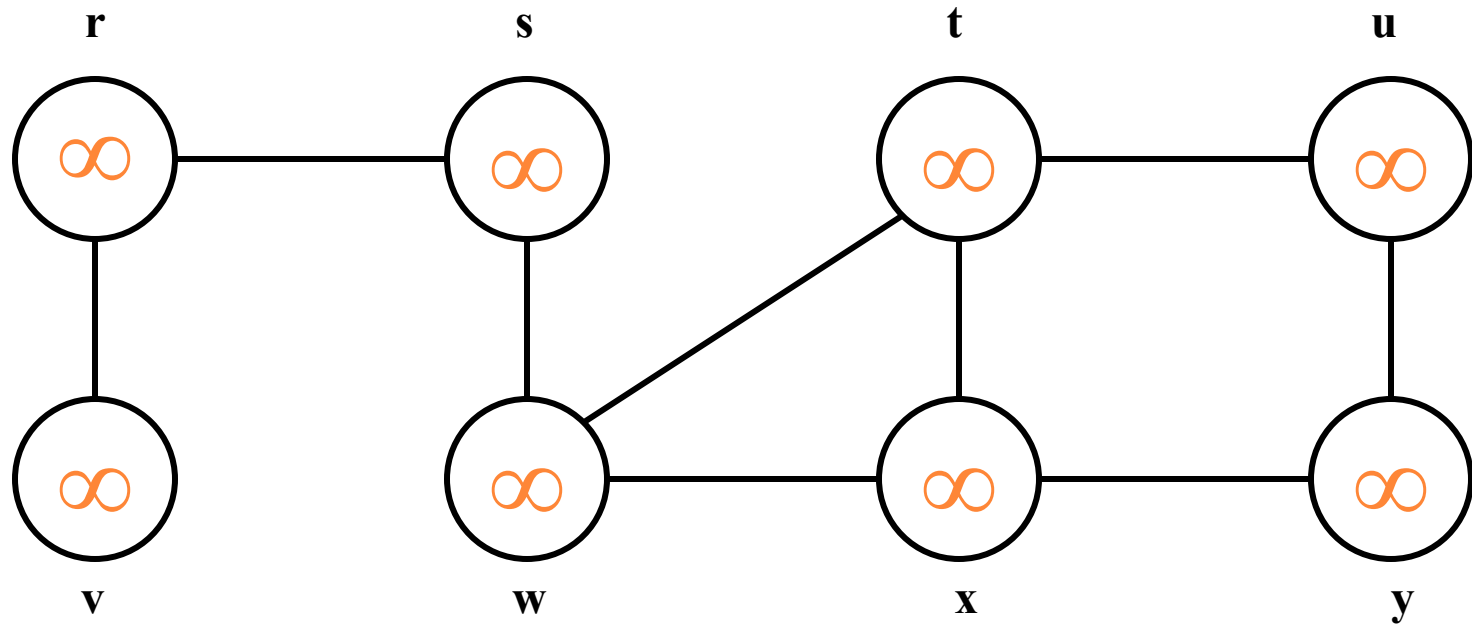
access source  $s$

white: undiscovered  
gray: discovered  
black: finished

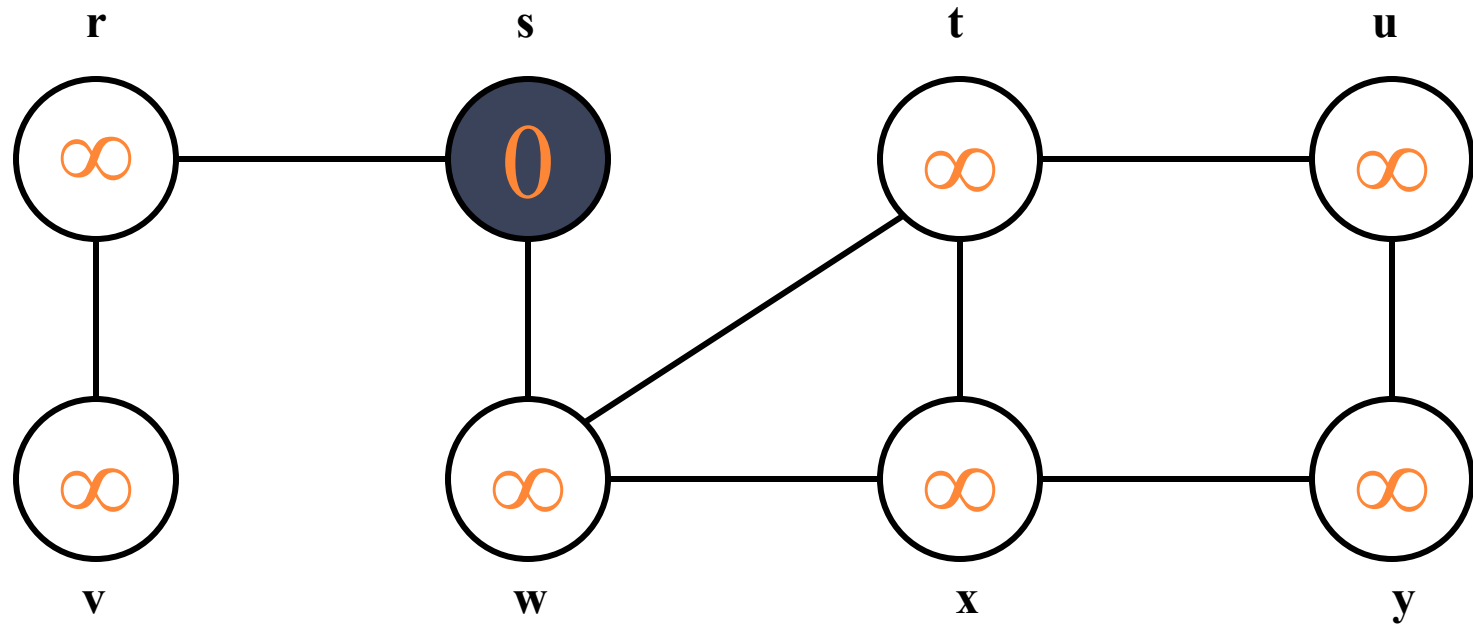
$Q$ : a queue of discovered vertices  
 $color[v]$ : color of  $v$   
 $d[v]$ : distance from  $s$  to  $v$   
 $\pi[u]$ : predecessor of  $v$



# BREADTH-FIRST SEARCH: EXAMPLE



# BREADTH-FIRST SEARCH: EXAMPLE

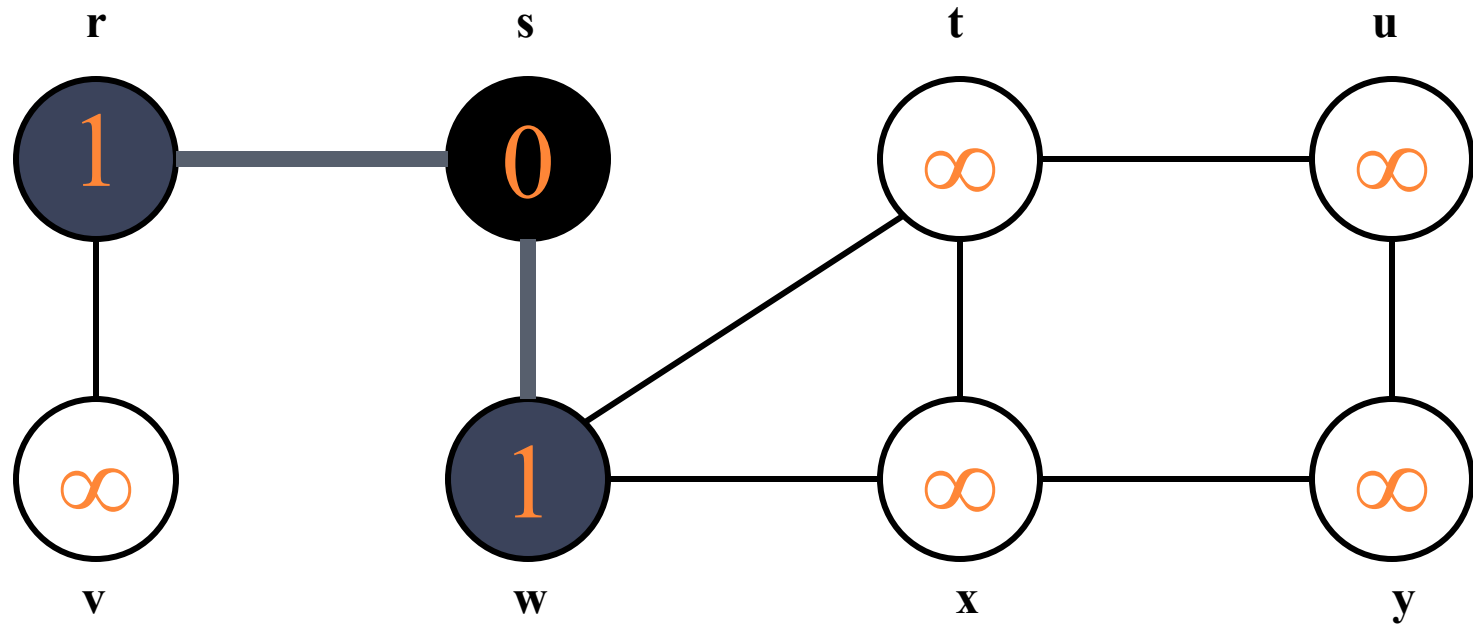


Q: s





# BREADTH-FIRST SEARCH: EXAMPLE

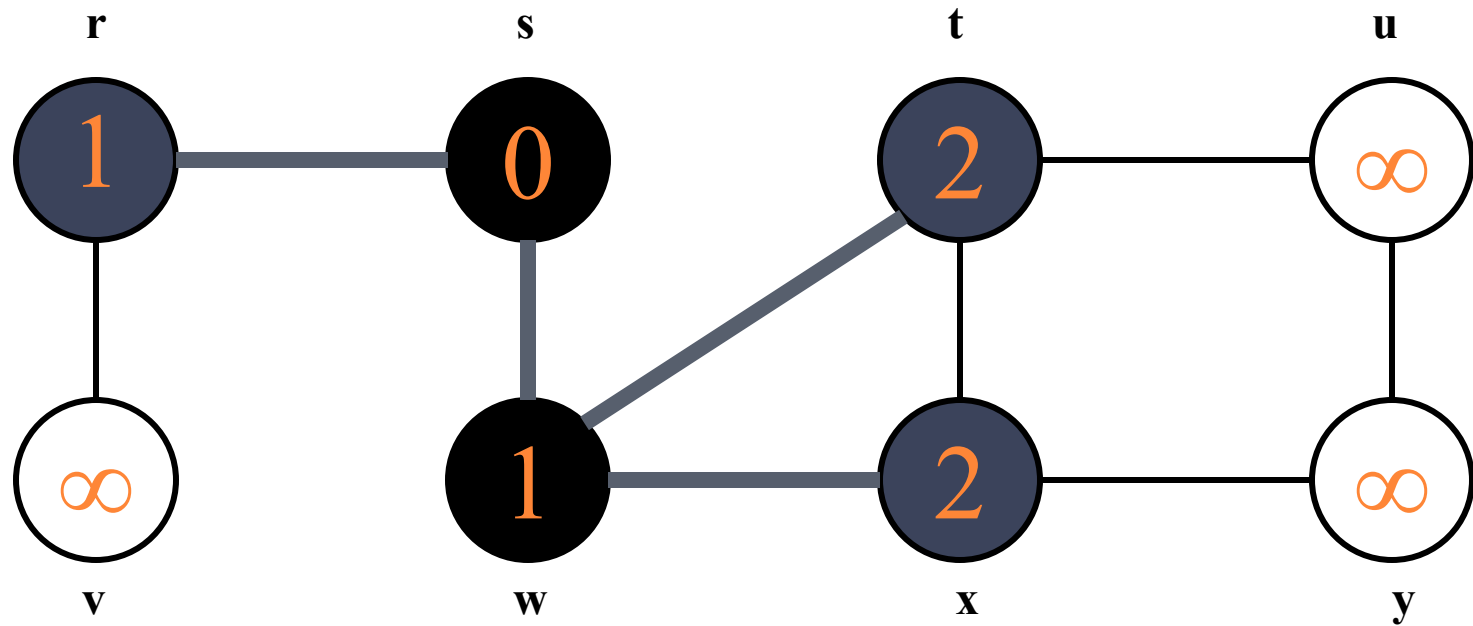


Q: 

|   |   |
|---|---|
| w | r |
|---|---|



# BREADTH-FIRST SEARCH: EXAMPLE

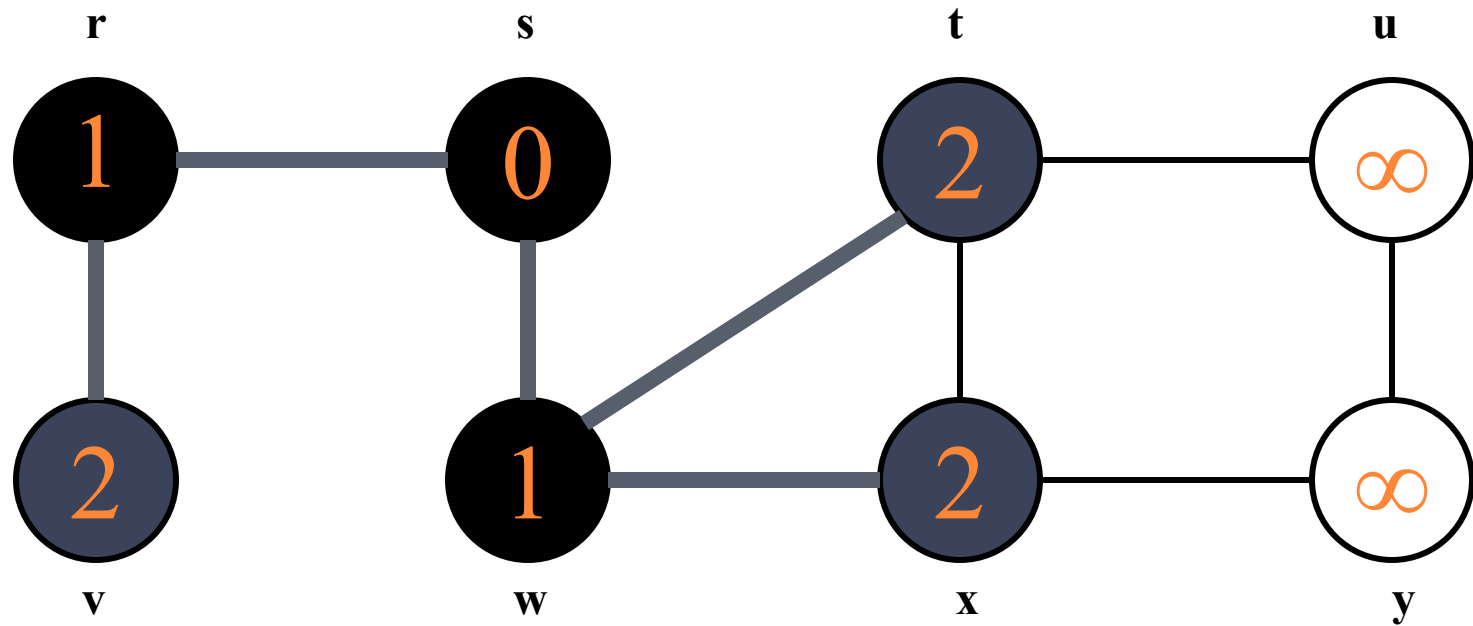


Q: 

|   |   |   |
|---|---|---|
| r | t | x |
|---|---|---|



# BREADTH-FIRST SEARCH: EXAMPLE

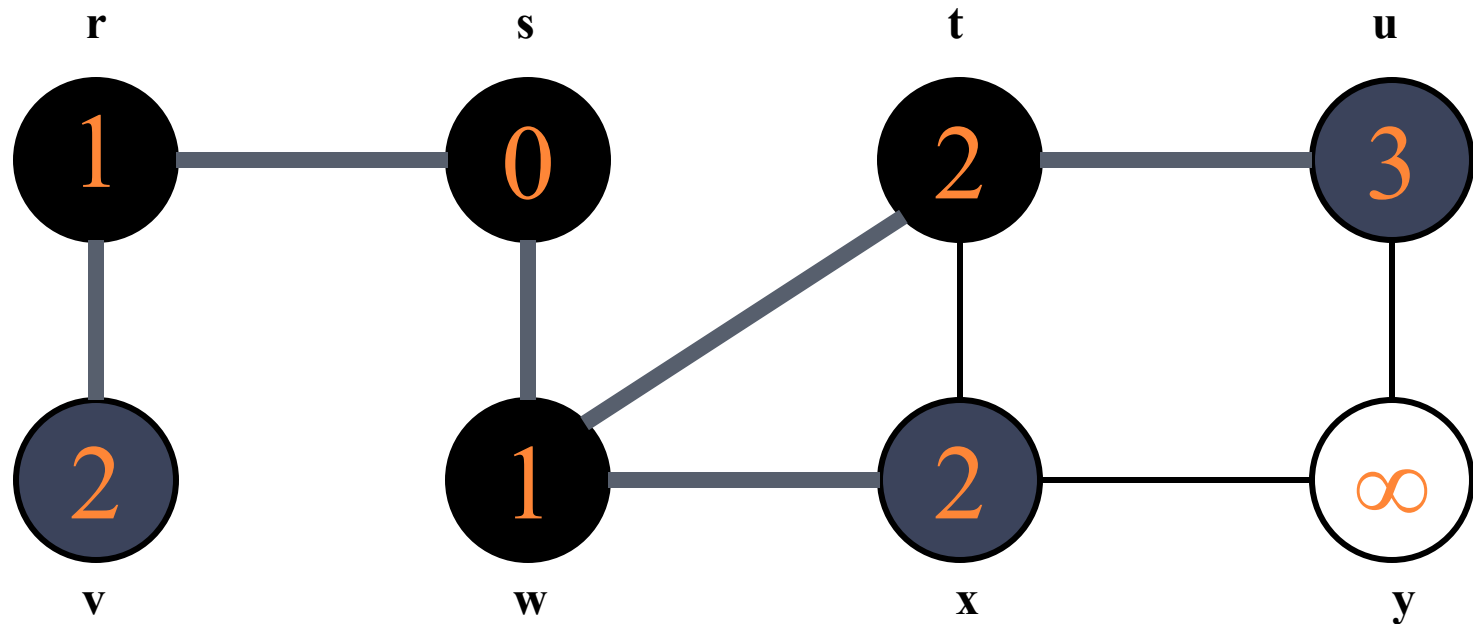


Q: 

|   |   |   |
|---|---|---|
| t | x | v |
|---|---|---|



# BREADTH-FIRST SEARCH: EXAMPLE

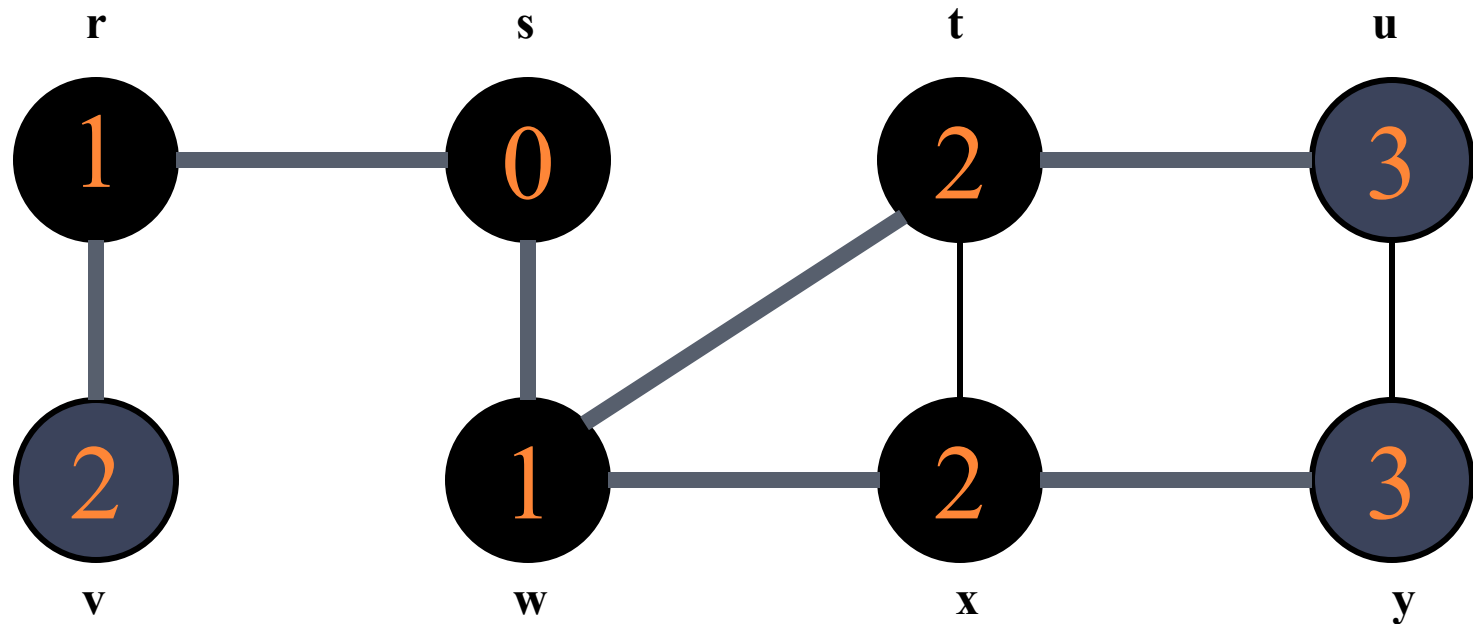


Q: 

|          |          |          |
|----------|----------|----------|
| <b>x</b> | <b>v</b> | <b>u</b> |
|----------|----------|----------|



# BREADTH-FIRST SEARCH: EXAMPLE

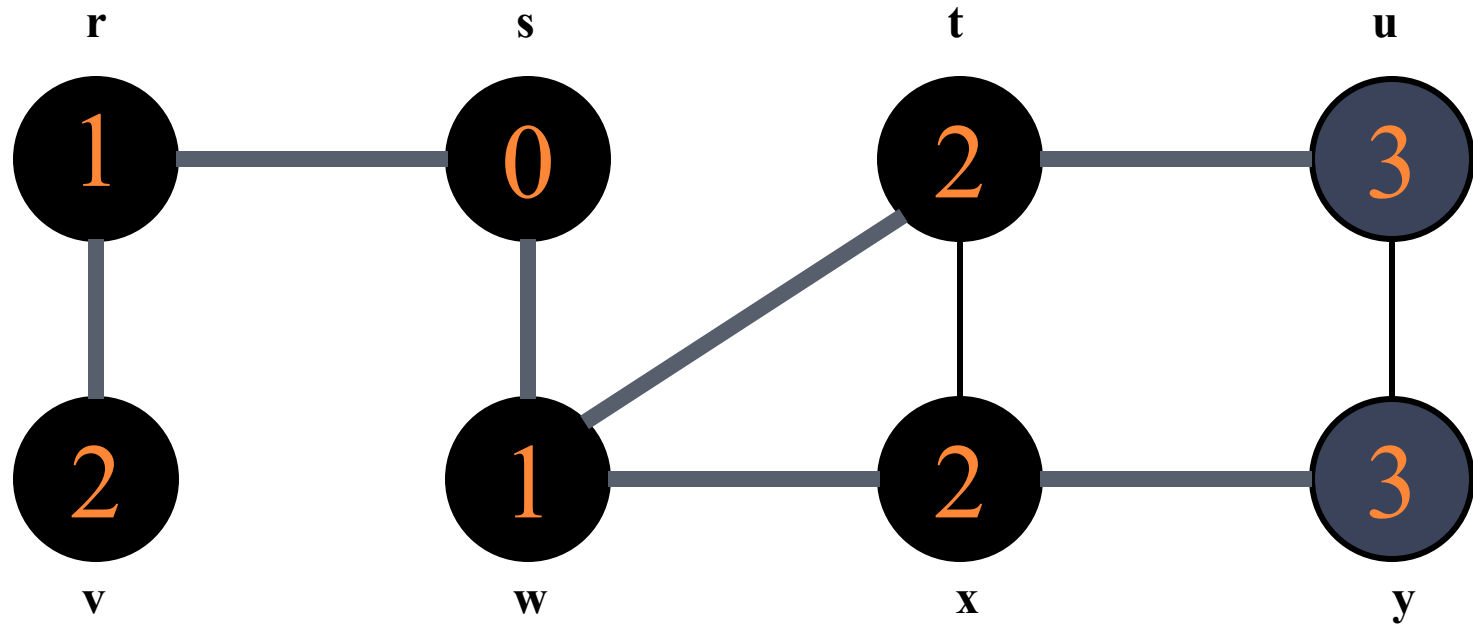


Q: 

|   |   |   |
|---|---|---|
| v | u | y |
|---|---|---|



# BREADTH-FIRST SEARCH: EXAMPLE

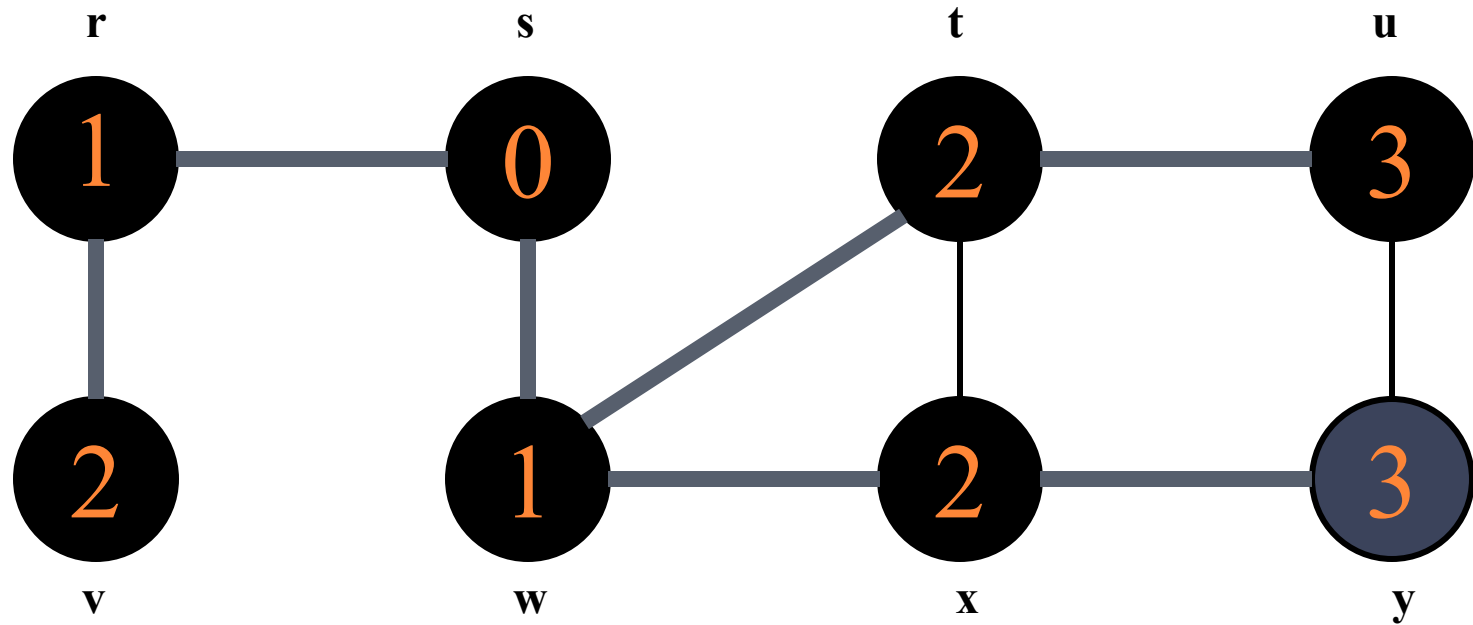


Q: 

|   |   |
|---|---|
| u | y |
|---|---|



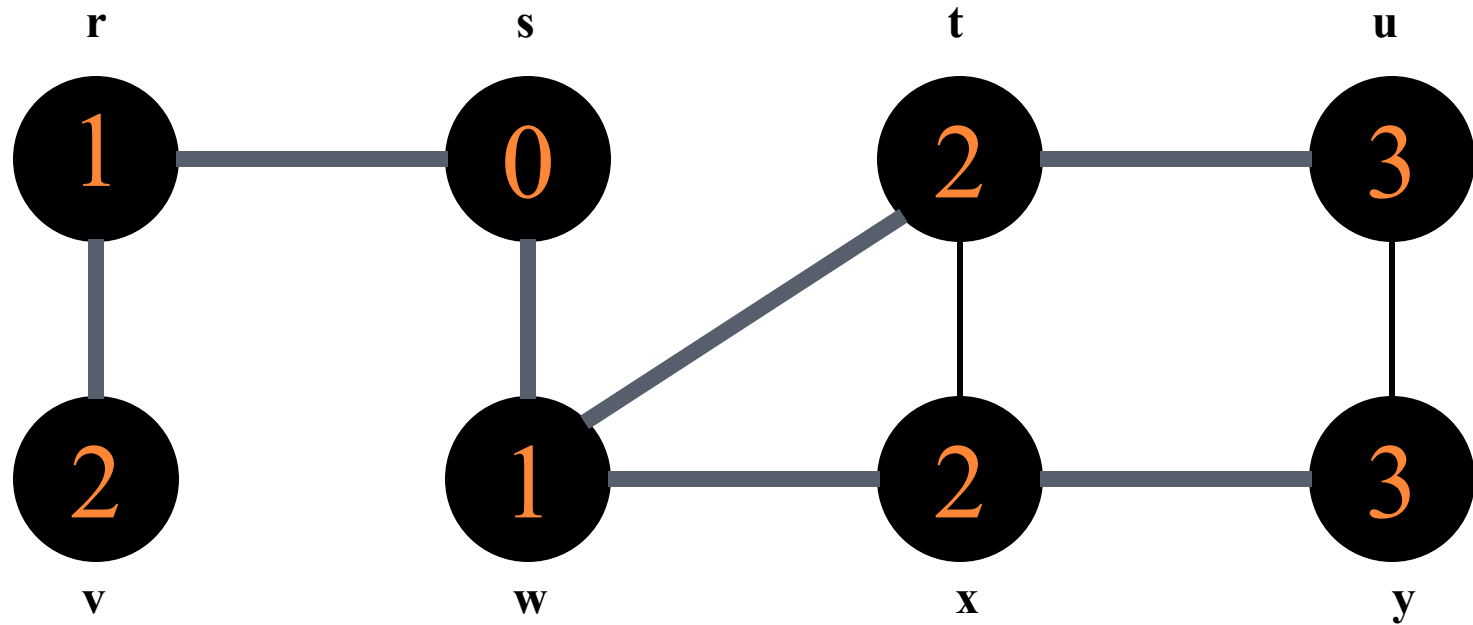
# BREADTH-FIRST SEARCH: EXAMPLE



Q: y



# BREADTH-FIRST SEARCH: EXAMPLE



**Q:**  $\emptyset$





# ANALYSIS OF BFS

- ? Initialization takes  $O(|V|)$ .
- ? Traversal Loop
  - After initialization, each vertex is enqueued and dequeued at most once, and each operation takes  $O(1)$ . So, total time for queuing is  $O(|V|)$ .
  - The adjacency list of each vertex is scanned at most once. The total time spent in scanning adjacency lists is  $O(|E|)$ .
- ? Summing up over all vertices  $\Rightarrow$  total running time of BFS is  $O(|V| + |E|)$



# BREADTH-FIRST TREE

- ? For a graph  $G = (V, E)$  with source  $s$ , the **predecessor subgraph** of  $G$  is  $G_\pi = (V_\pi, E_\pi)$  where
- $V_\pi = \{v \in V : \pi[v] \neq \text{nil}\} \sqcup \{s\}$
  - $E_\pi = \{(\pi[v], v) \in E : v \in V_\pi - \{s\}\}$
- ? The predecessor subgraph  $G_\pi$  is a **breadth-first tree** if:
- $V_\pi$  consists of the vertices reachable from  $s$  and
  - for all  $v \in V_\pi$ , there is a unique simple path from  $s$  to  $v$  in  $G_\pi$  that is also a shortest path from  $s$  to  $v$  in  $G$ .
- ? The edges in  $E_\pi$  are called **tree edges**.  
 $|E_\pi| = |V_\pi| - 1$ .



# DEPTH-FIRST SEARCH (DFS)

- ? Explore edges out of the most recently discovered vertex  $v$ .
- ? When all edges of  $v$  have been explored, backtrack to explore other edges leaving the vertex from which  $v$  was discovered (its *predecessor*).
- ? “Search as deep as possible first.”
- ? Continue until all vertices reachable from the original source are discovered.
- ? If any undiscovered vertices remain, then one of them is chosen as a new source and search is repeated from that source.



## DFS - 1

parent = {s:none}

DFS\_Visit(s, Adj.s)

for v in Adj[s]

if v is not in parent

parent[v] =s

DFS\_Visit(v, Adj.v)

DFS (V, Adj)

parent ={}

for s in V

if s is not in parent

parent[s] = none

DFS\_Visit(s,Adj.s)



# DEPTH-FIRST SEARCH

? **Input:**  $G = (V, E)$ , directed or undirected.  
No source vertex given!

? **Output:**

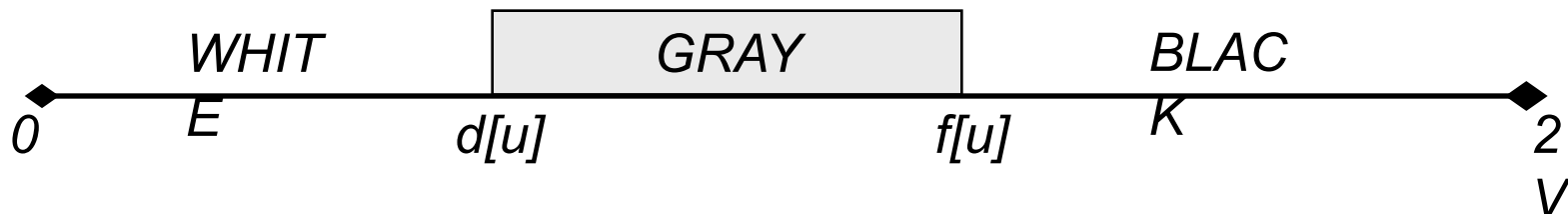
- 2 timestamps on each vertex.
  - ?  $d[v] = \textit{discovery time}$  ( $v$  turns from white to gray)
  - ?  $f[v] = \textit{finishing time}$  ( $v$  turns from gray to black)
- $\pi[v]$  : predecessor of  $v = u$ , such that  $v$  was discovered during the scan of  $u$ 's adjacency list.
- Depth-first forest



# DEPTH-FIRST SEARCH

- ? Coloring scheme for vertices as BFS.
- A vertex is “**discovered**” the first time it is encountered during the search.
  - A vertex is “**finished**” if it is a leaf node or all vertices adjacent to it have been finished.
  - White before **discovery**, gray while processing and black when **finished** processing

$$1 \leq d[u] < f[u] \leq 2|V|$$



# PSEUDOCODE

## DFS( $G$ )

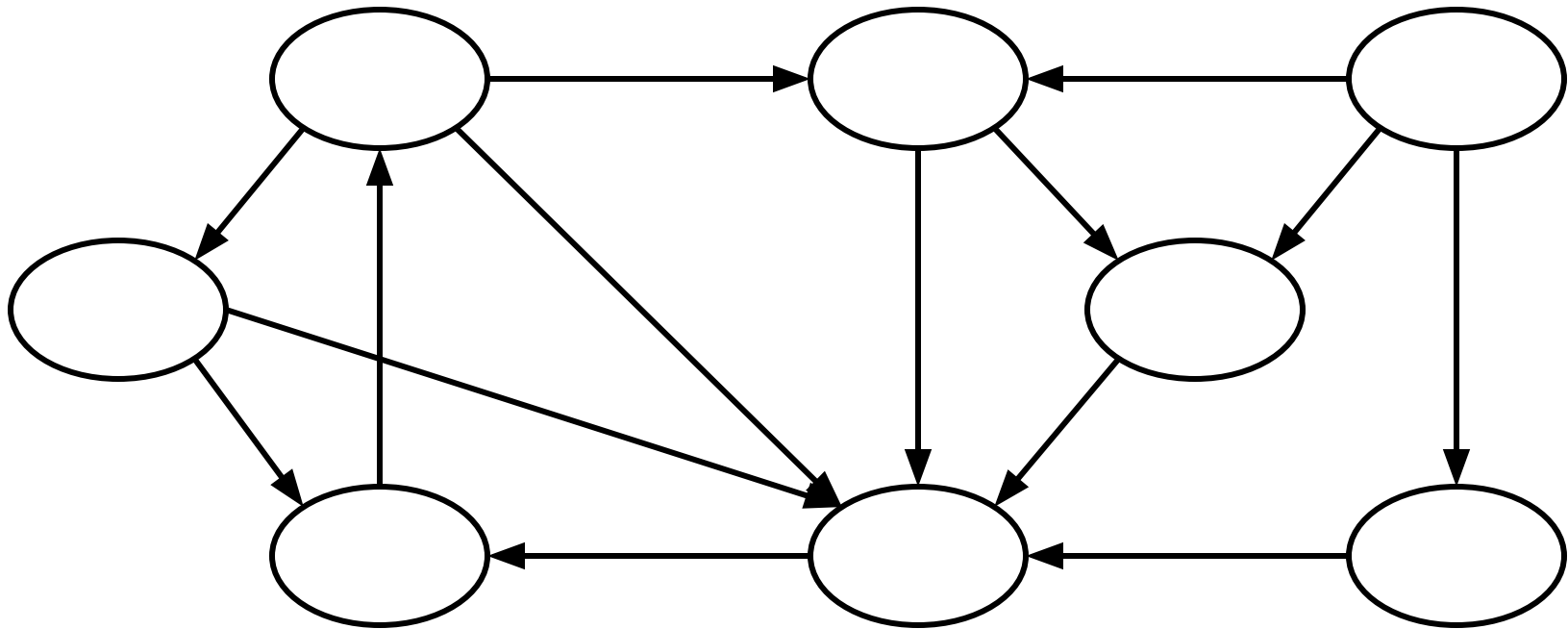
1. **for** each vertex  $u \in V[G]$
2.     **do**  $color[u] \leftarrow \text{white}$
3.      $\pi[u] \leftarrow \text{NIL}$
4.  $time \leftarrow 0$
5. **for** each vertex  $u \in V[G]$
6.     **do if**  $color[u] = \text{white}$
7.         **then** DFS-Visit( $u$ )

## DFS-Visit( $u$ )

1.      $color[u] \leftarrow \text{GRAY}$  // White vertex  $u$   
          has been discovered
2.      $time \leftarrow time + 1$
3.      $d[u] \leftarrow time$
4.     **for** each  $v \in Adj[u]$
5.         **do if**  $color[v] = \text{WHITE}$
6.             **then**  $\pi[v] \leftarrow u$
7.             DFS-Visit( $v$ )
8.      $color[u] \leftarrow \text{BLACK}$  // Blacken  $u$ ;  
          it is finished.
9.      $f[u] \leftarrow time \leftarrow time + 1$

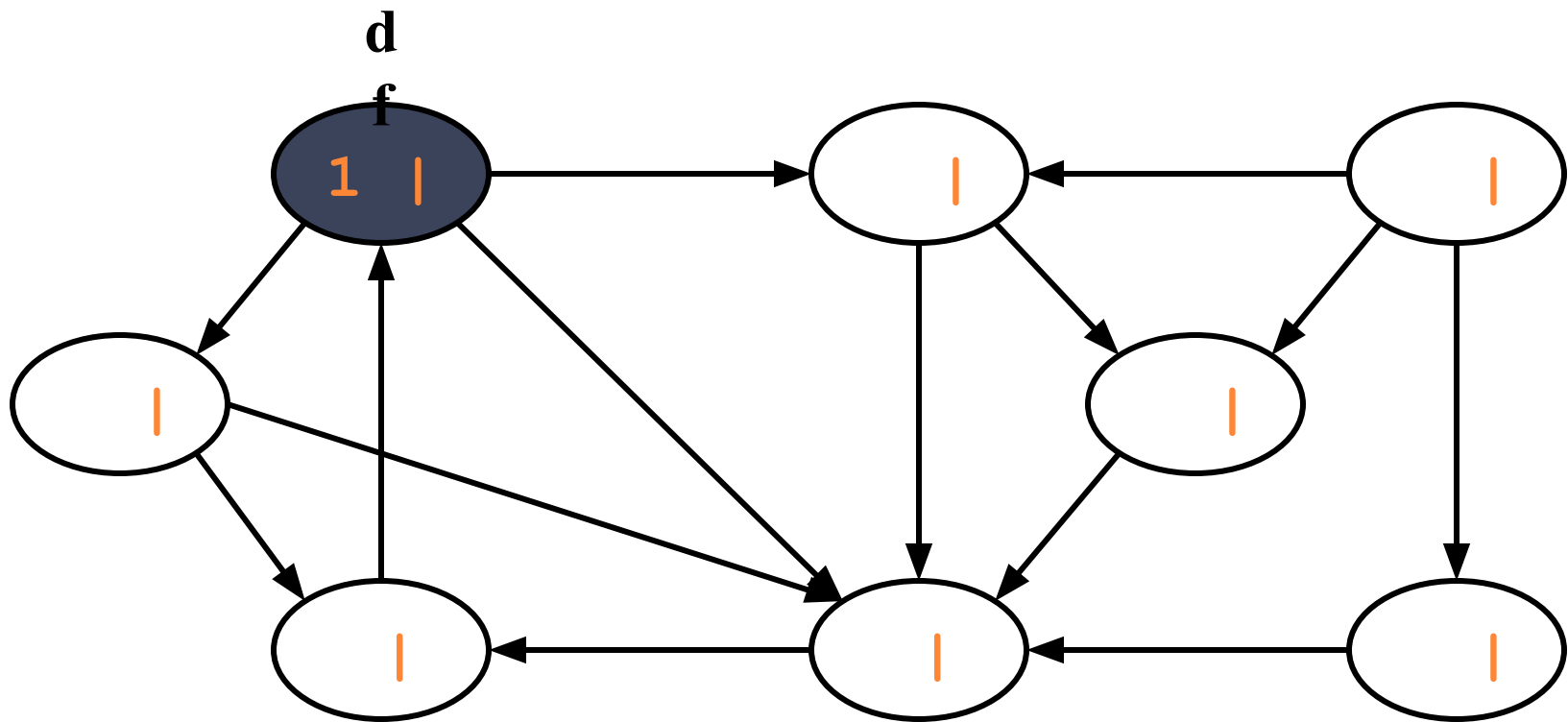
Uses a global timestamp *time*.

# DFS EXAMPLE

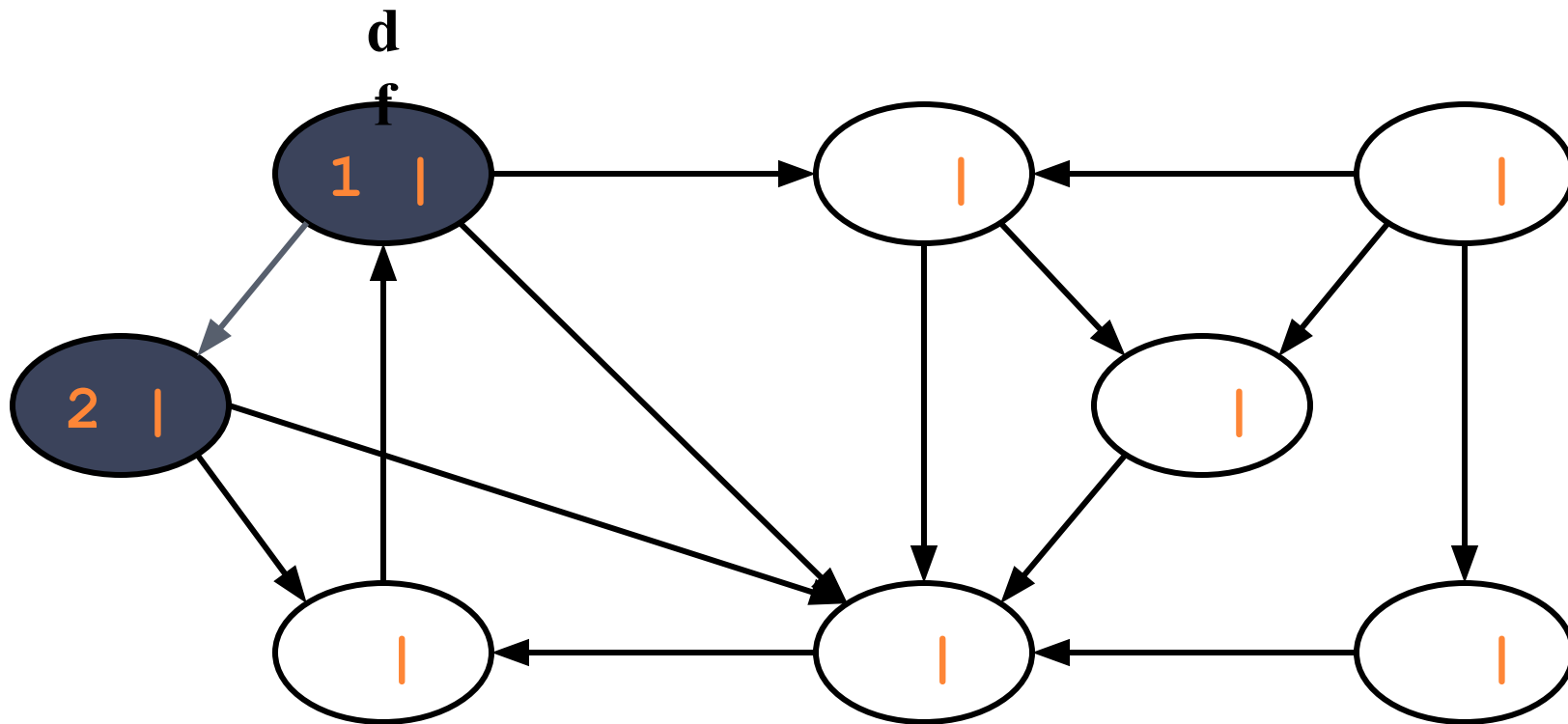




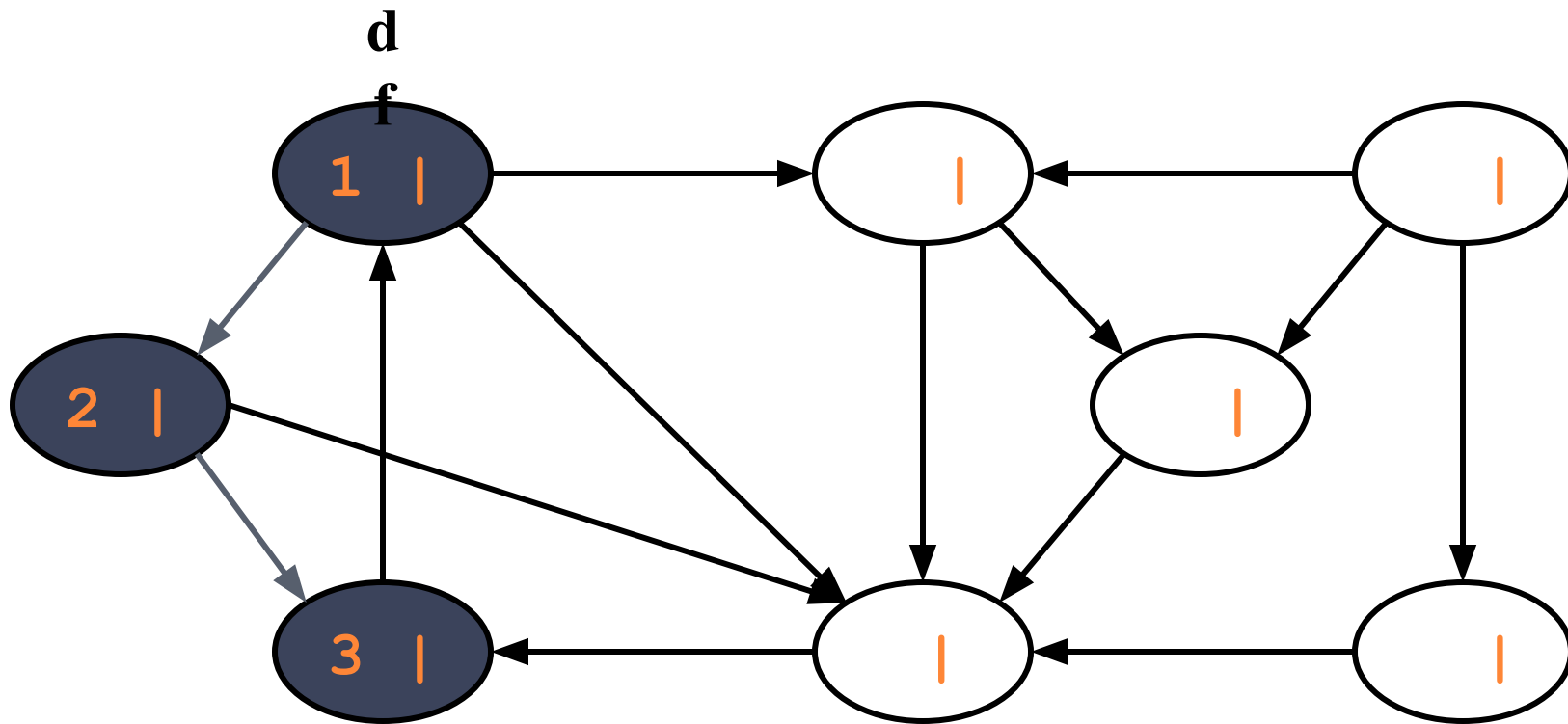
# DFS EXAMPLE



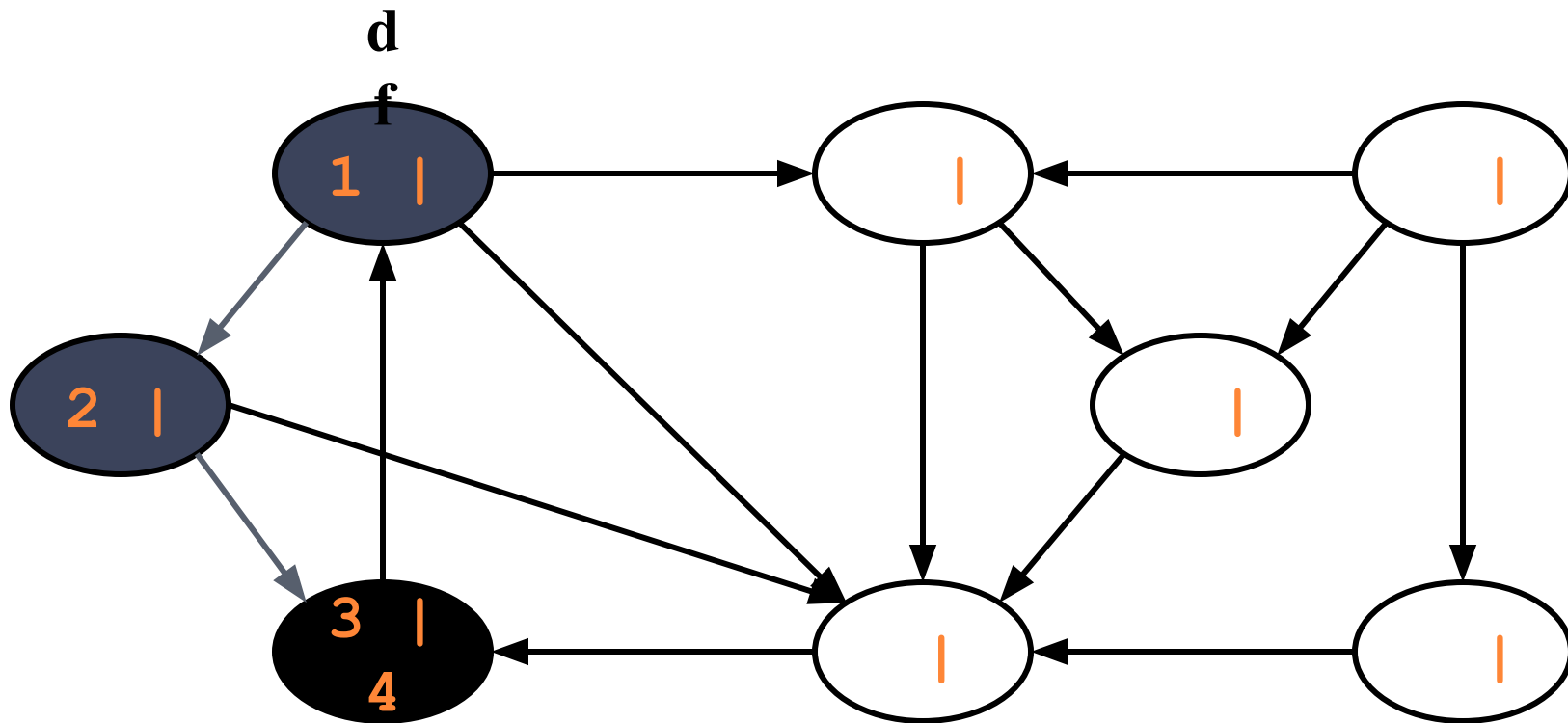
# DFS EXAMPLE



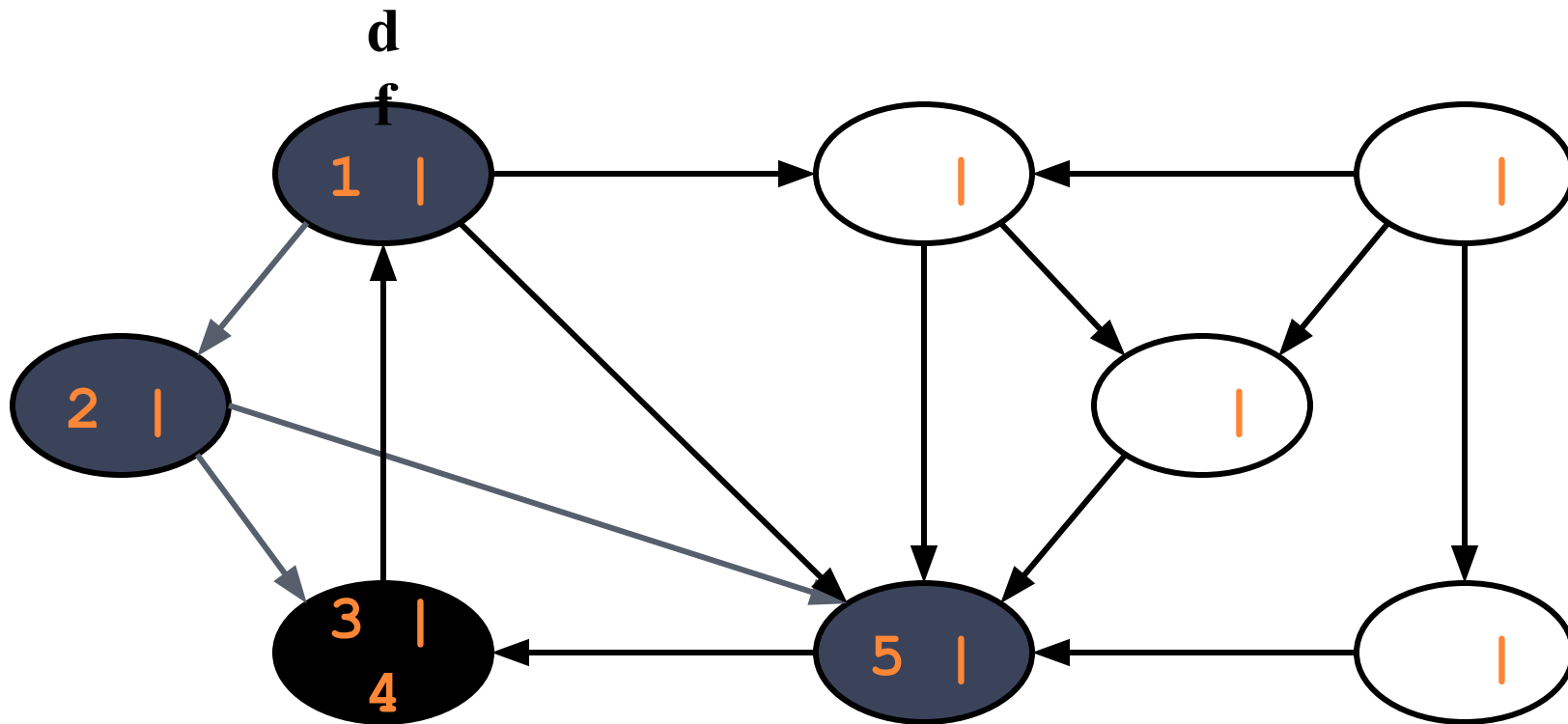
# DFS EXAMPLE



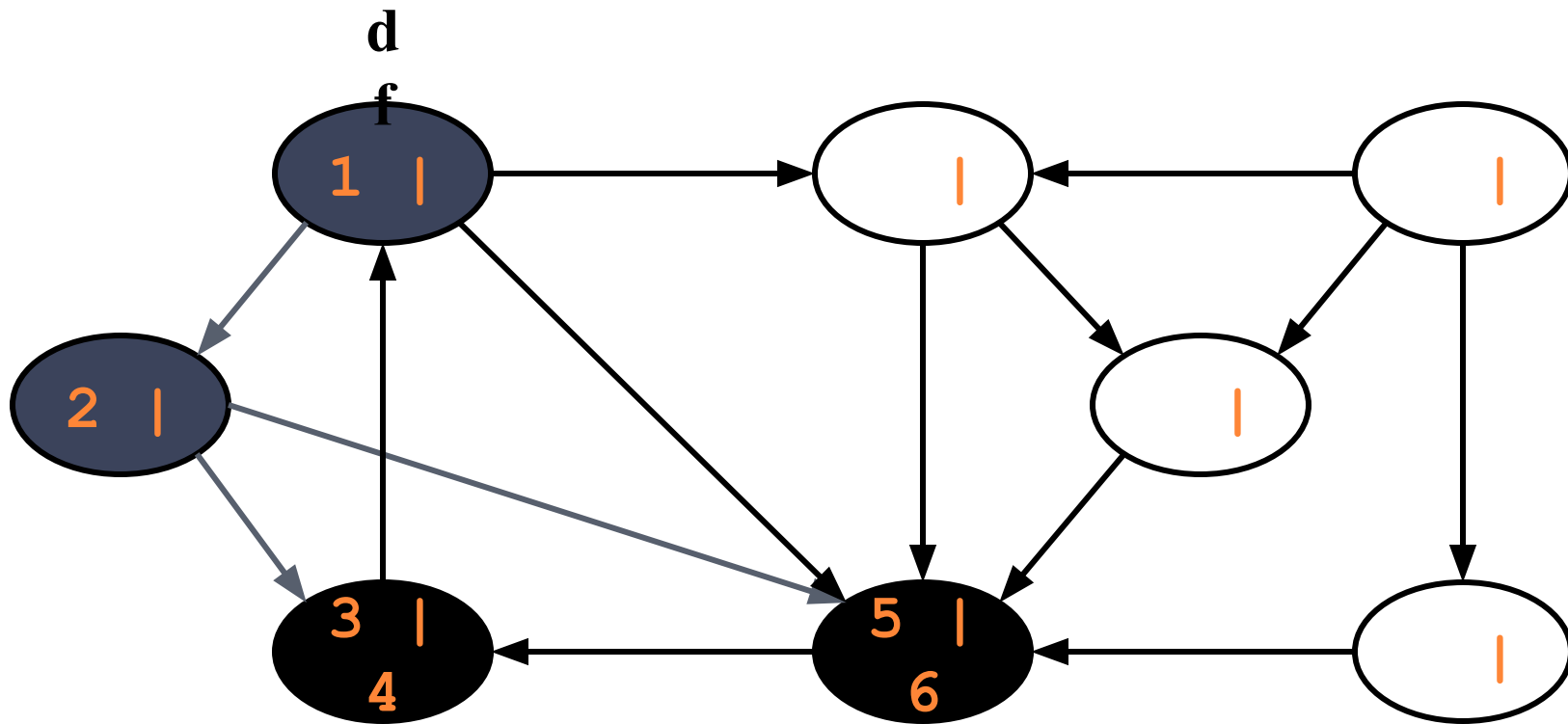
# DFS EXAMPLE



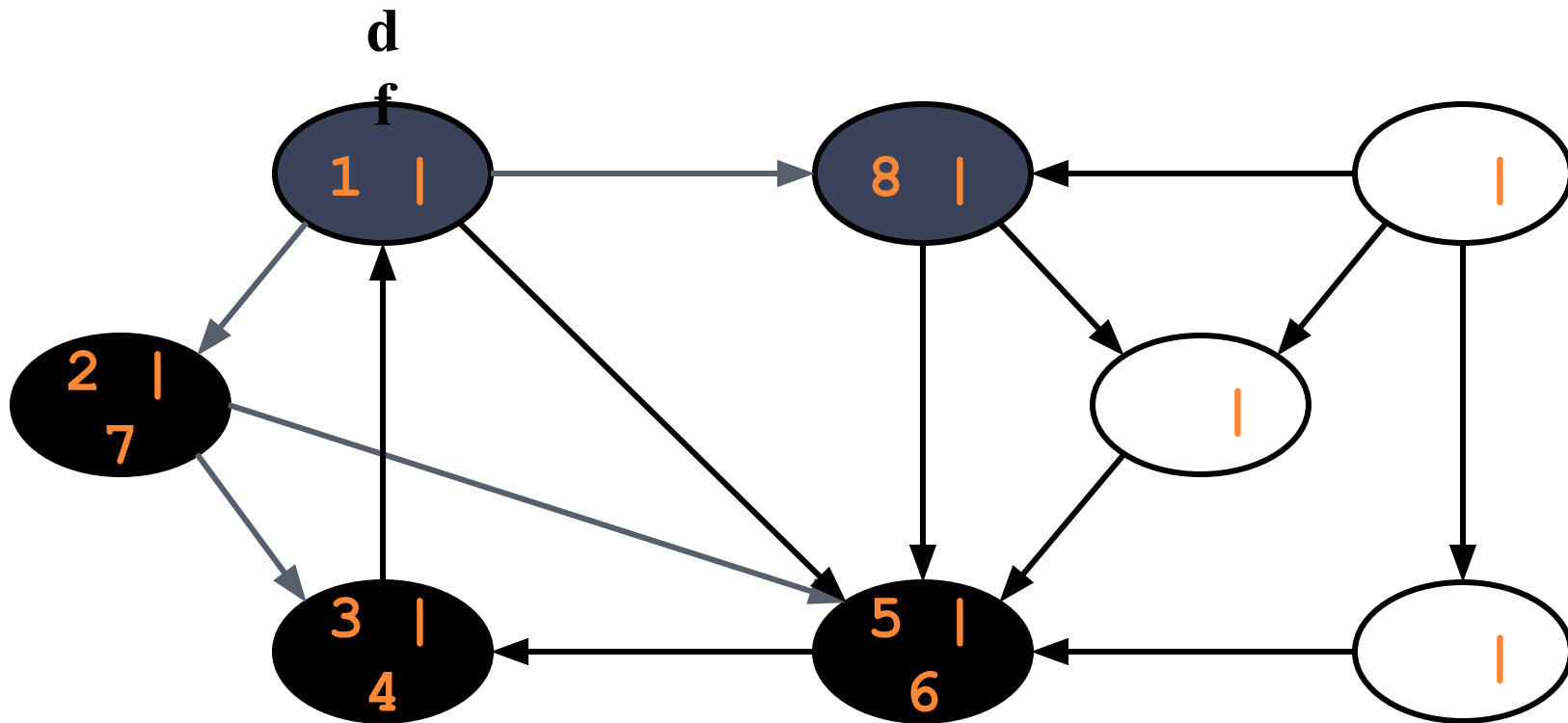
# DFS EXAMPLE



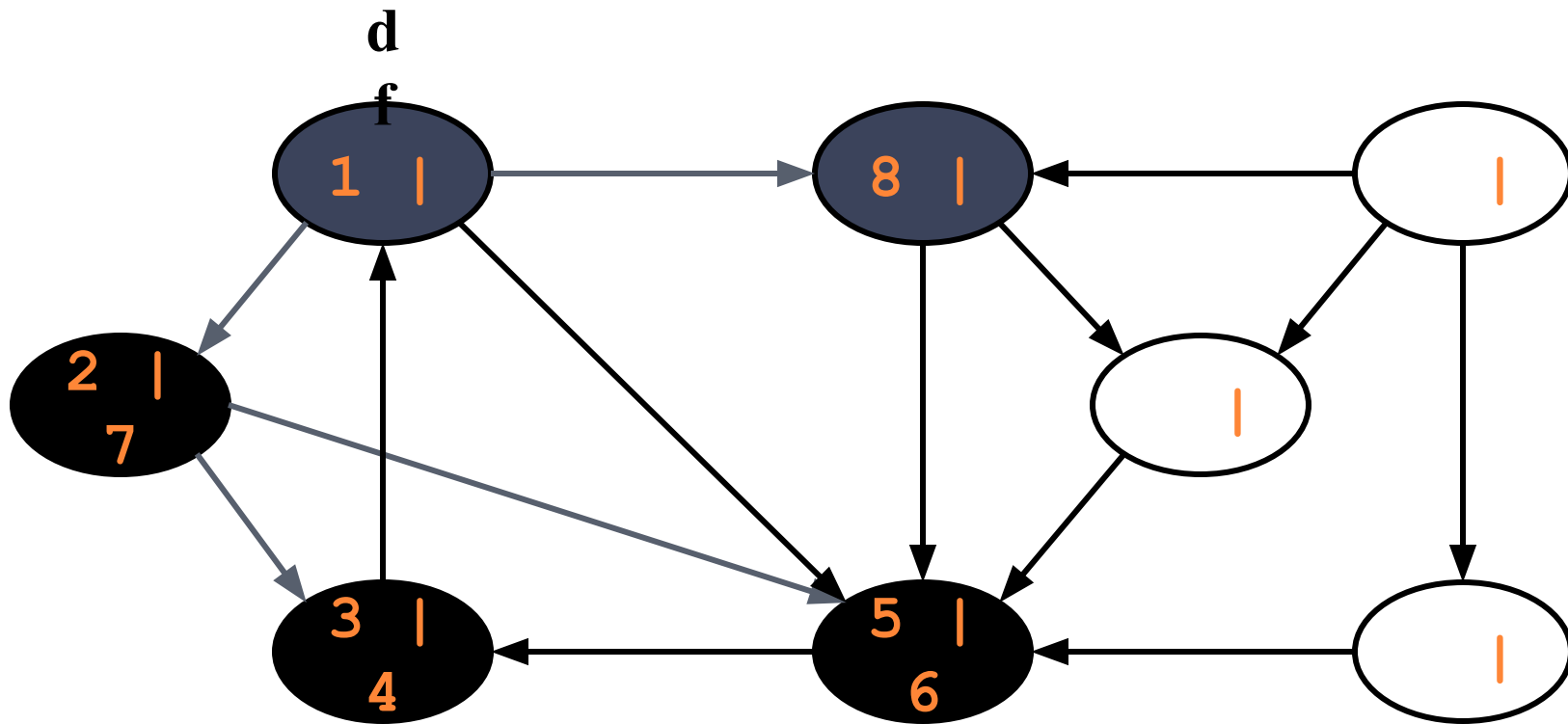
# DFS EXAMPLE



# DFS EXAMPLE

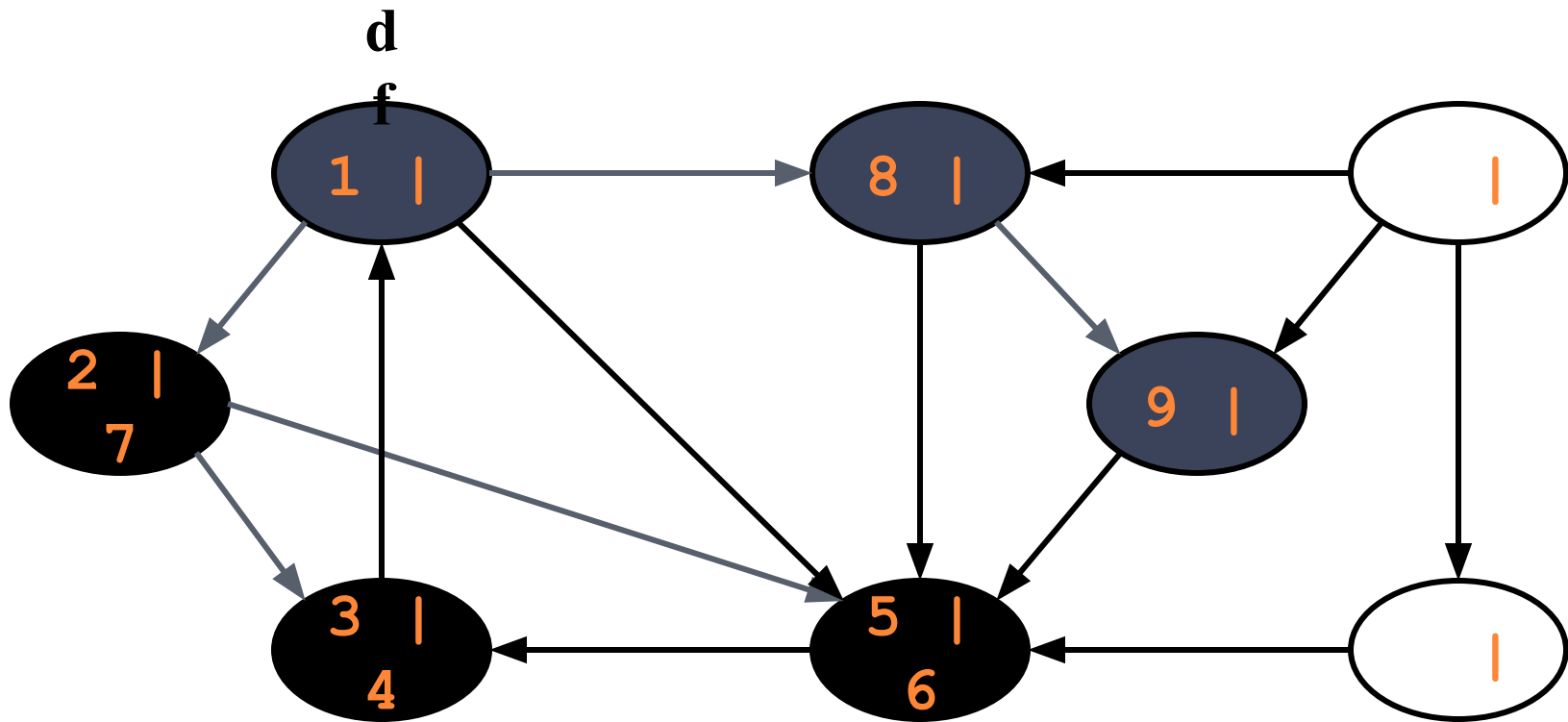


# DFS EXAMPLE

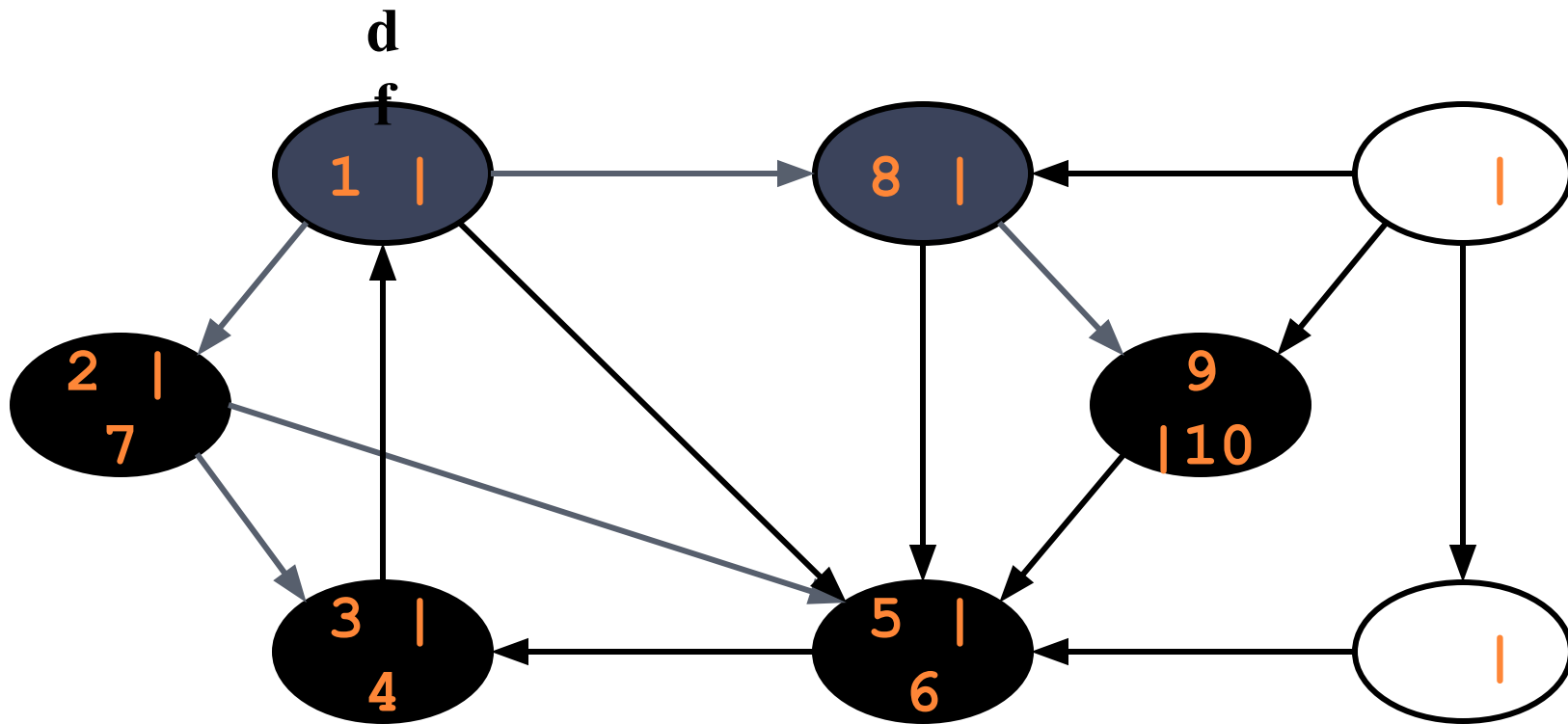




# DFS EXAMPLE



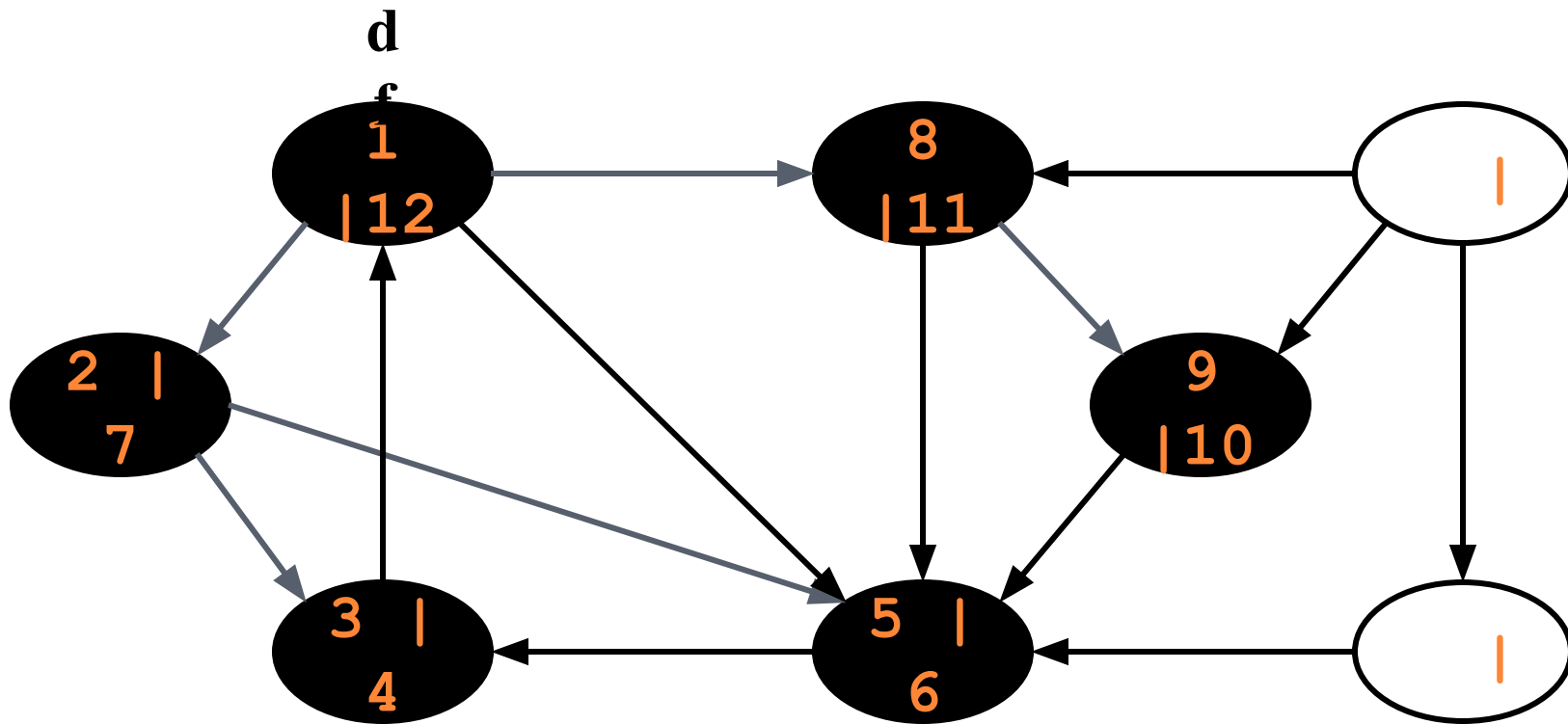
# DFS EXAMPLE



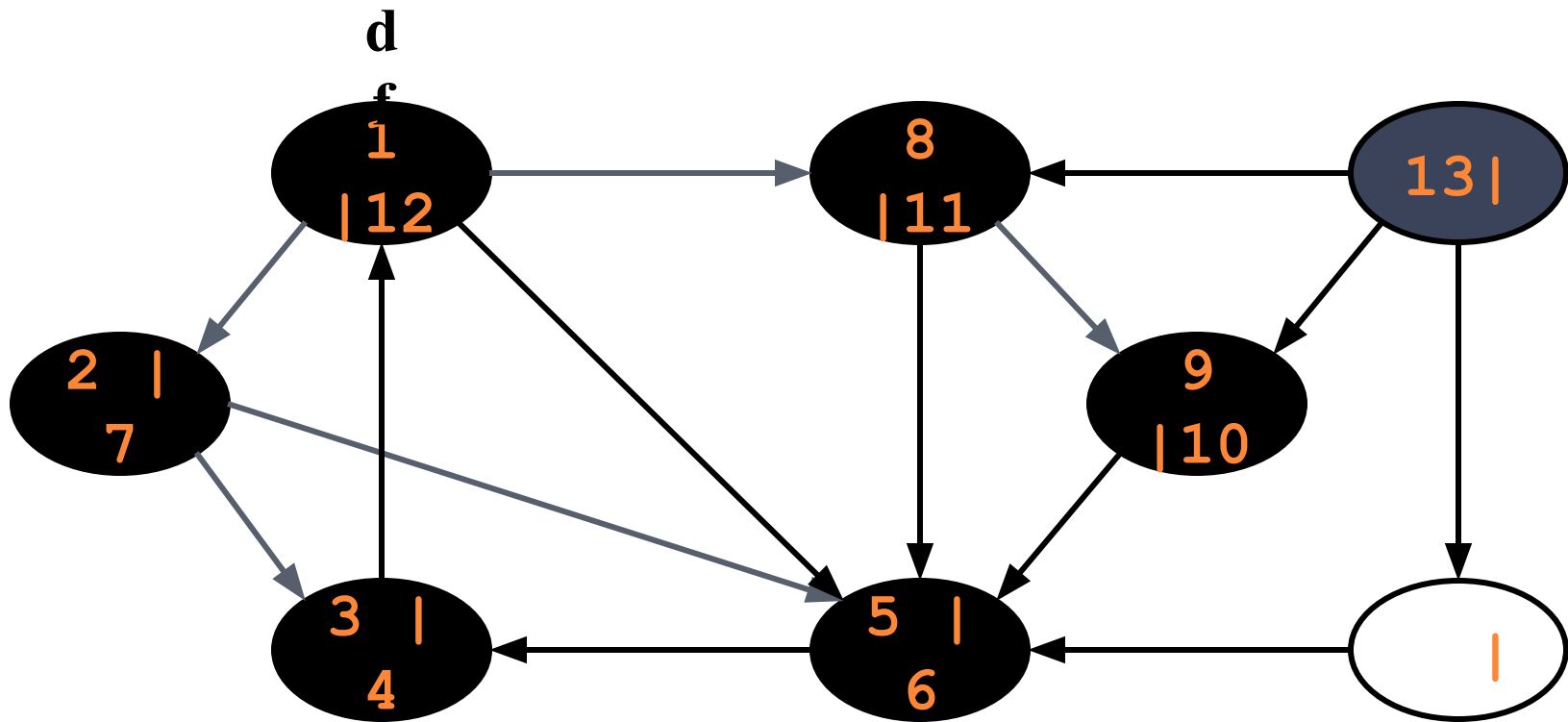
## DFS EXAMPLE



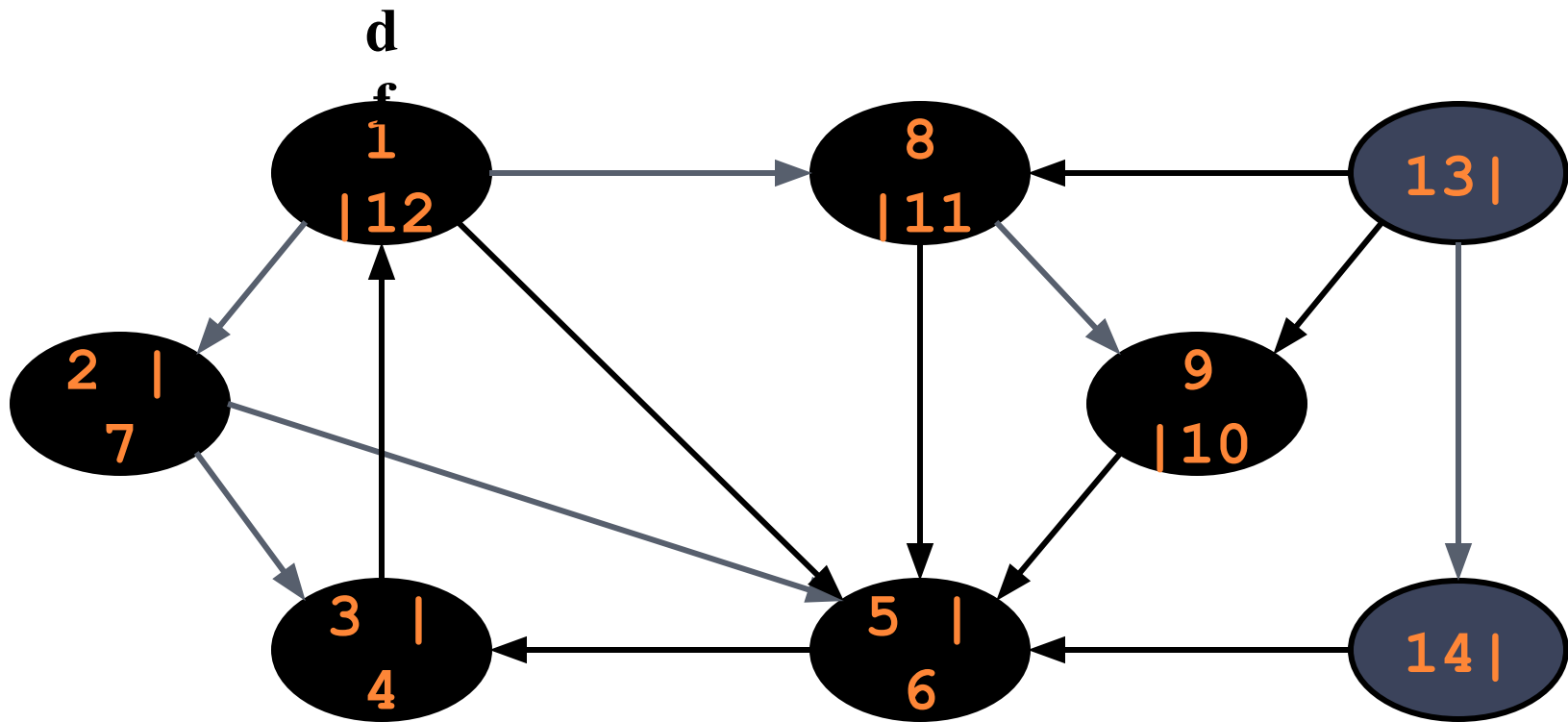
# DFS EXAMPLE



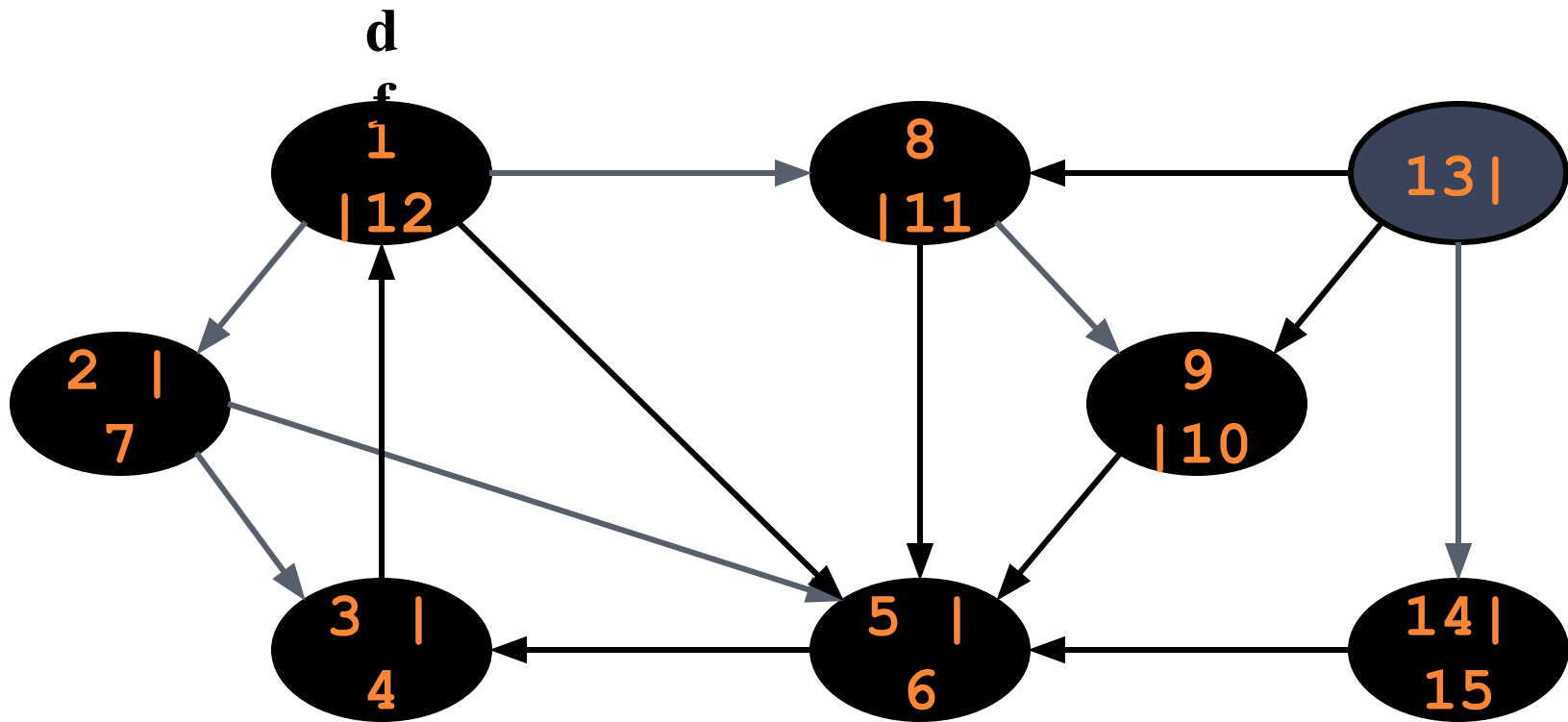
# DFS EXAMPLE



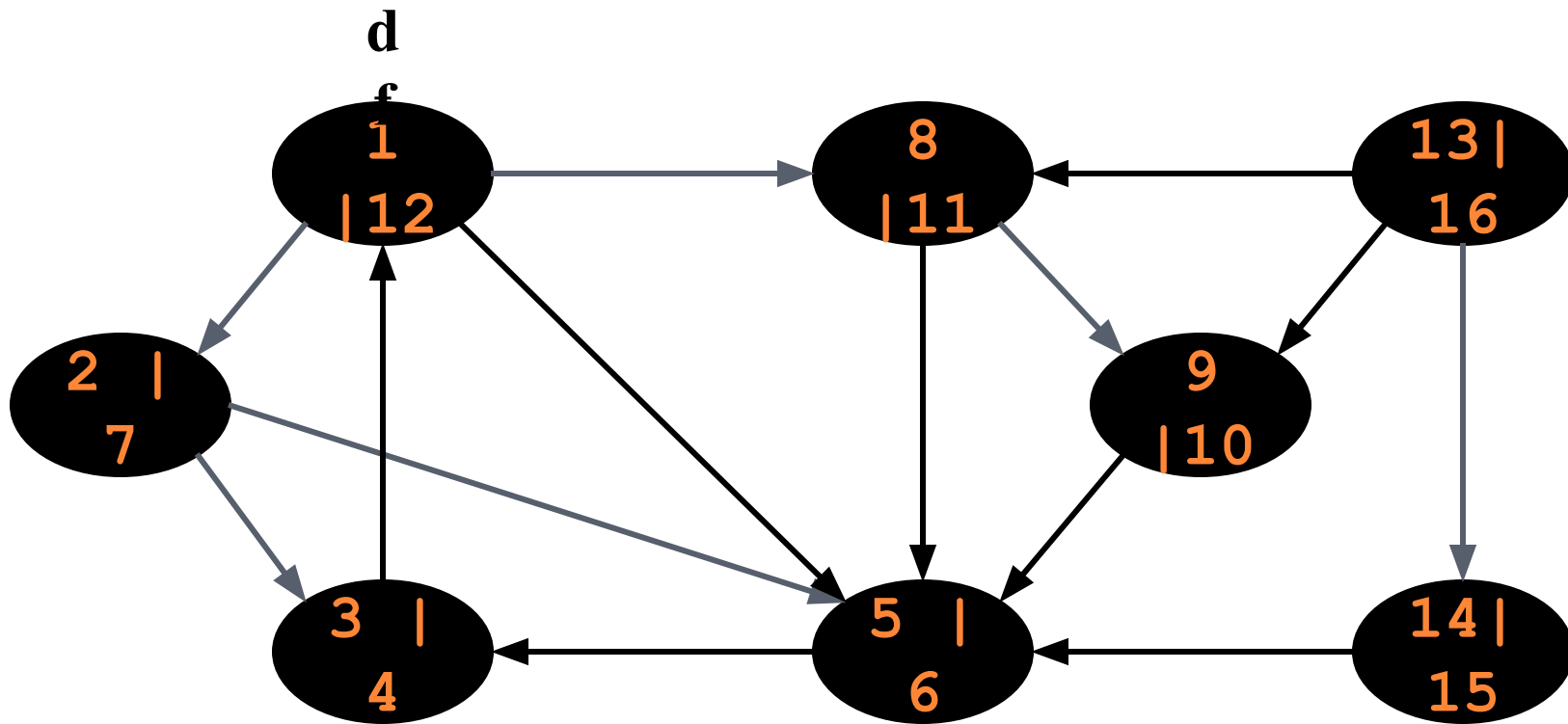
# DFS EXAMPLE



# DFS EXAMPLE



# DFS EXAMPLE





## ANALYSIS OF DFS

- ? Loops on lines 1-3 & 5-7 take  $\Theta(V)$  time, excluding time to execute DFS-Visit.
- ? DFS-Visit is called once for each white vertex  $v \in V$  when it's painted gray the first time. Lines 4-7 of DFS-Visit is executed  $|Adj[v]|$  times. The total cost of executing DFS-Visit is  $\sum_{v \in V} |Adj[v]| = \Theta(E)$
- ? Total running time of DFS is  $\Theta(|V| + |E|)$ .



# DEPTH-FIRST TREES

- ? Predecessor subgraph defined slightly different from that of BFS.
- ? The predecessor subgraph of DFS is  $G_\pi = (V, E_\pi)$  where  $E_\pi = \{(\pi[v], v) : v \in V \text{ and } \pi[v] \neq \text{nil}\}$ .
  - How does it differ from that of BFS?
  - The predecessor subgraph  $G_\pi$  forms a *depth-first forest* composed of several *depth-first trees*. The edges in  $E_\pi$  are called *tree edges*.



# TIME-STAMP STRUCTURE IN DFS

? There is also a nice structure to the time stamps, which is referred to as *Parenthesis Structure*.

## **Theorem 22.7**

For all  $u, v$ , exactly one of the following holds:

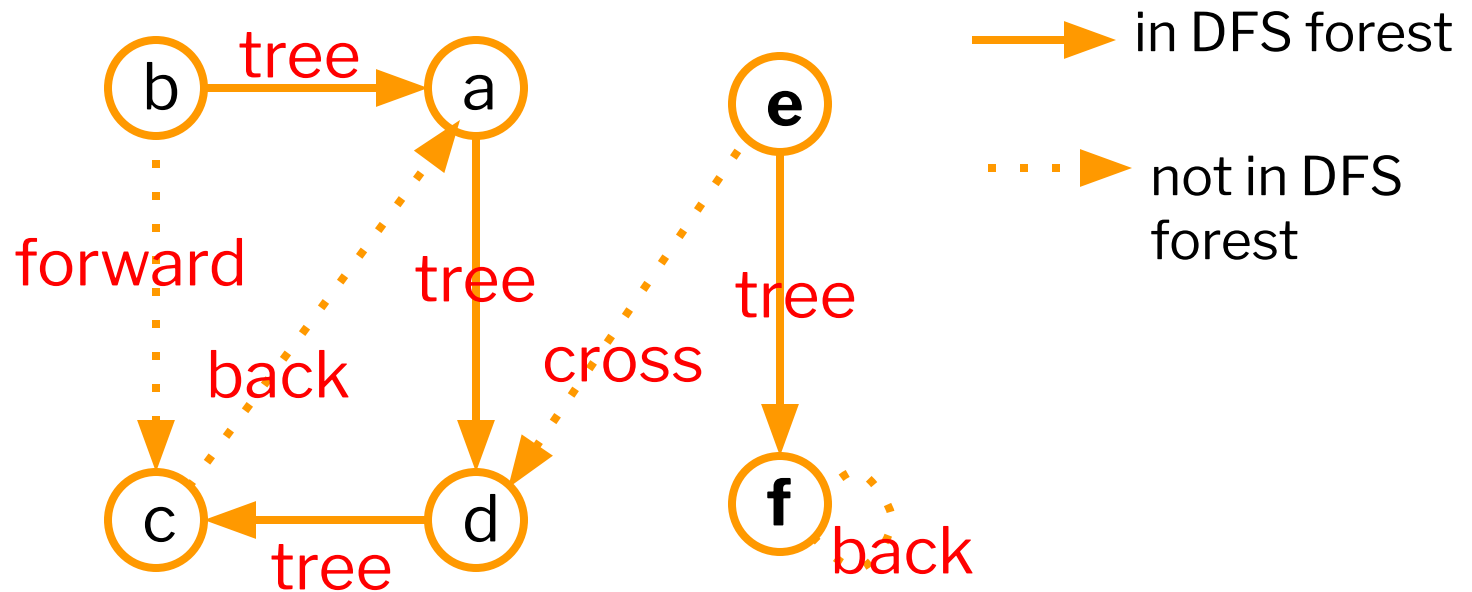
1.  $d[u] < f[u] < d[v] < f[v]$  or  $d[v] < f[v] < d[u] < f[u]$  and neither  $u$  nor  $v$  is a descendant of the other.
2.  $d[u] < d[v] < f[v] < f[u]$  and  $v$  is a descendant of  $u$ .
3.  $d[v] < d[u] < f[u] < f[v]$  and  $u$  is a descendant of  $v$ .



## DFS: KINDS OF EDGES

- ? Consider a directed graph  $G = (V, E)$ . After a DFS of graph  $G$  we can put each edge into one of four classes:
- A **tree edge** is an edge in a DFS-tree.
  - A **back edge** connects a vertex to an ancestor in a DFS-tree. Note that a self-loop is a back edge.
  - A **forward edge** is a non-tree edge that connects a vertex to a descendent in a DFS-tree.
  - A **cross edge** is any other edge in graph  $G$ . It connects vertices in two different DFS-trees or two vertices in the same DFS-tree neither of which is the ancestor of the other.
- 

# EXAMPLE OF CLASSIFYING EDGES



# CLASSIFYING EDGES OF A DIGRAPH

? (u, v) is:

- Tree edge – if v is white
- Back edge – if v is gray
- Forward or cross - if v is black

? (u, v) is:

- Forward edge – if v is black and  $d[u] < d[v]$  (v was discovered after u)
- Cross edge – if v is black and  $d[u] > d[v]$  (u discovered after v)



# DFS: KINDS OF EDGES

**DFS-Visit( $u$ )**       $\triangleright$  with edge classification.  $G$  must be a directed graph

```
1.  color[u]  $\leftarrow$  GRAY
2.  time  $\leftarrow$  time + 1
3.  d[u]  $\leftarrow$  time
4.  for each vertex  $v$  adjacent to  $u$ 
5.      do if color[v]  $\leftarrow$  BLACK
6.          then if d[u] < d[v]
7.              then Classify ( $u, v$ ) as a forward edge
8.              else Classify ( $u, v$ ) as a cross edge
9.          if color[v]  $\leftarrow$  GRAY
10.             then Classify ( $u, v$ ) as a back edge
11.         if color[v]  $\leftarrow$  WHITE
12.             then  $\pi[v] \leftarrow u$ 
13.                 Classify ( $u, v$ ) as a tree edge
14.                 DFS-Visit( $v$ )
15.  color[u]  $\leftarrow$  BLACK
16.  time  $\leftarrow$  time + 1
17.  f[u]  $\leftarrow$  time
```



## DFS: KINDS OF EDGES

? Suppose  $G$  be an undirected graph, then we have following edge classification:

- **Tree Edge** an edge connects a vertex with its parent.
- **Back Edge** a non-tree edge connects a vertex with an ancestor.
- **Forward Edge** There is no forward edges because they become back edges when considered in the opposite direction.
- **Cross Edge** There cannot be any cross edge because every edge of  $G$  must connect an ancestor with a descendant.





# SOME APPLICATIONS OF BFS AND DFS

## ? BFS

- To find the shortest path from a vertex  $s$  to a vertex  $v$  in an unweighted graph
- To find the length of such a path
- Find the bipartiteness of a graph.

## ? DFS

- To find a path from a vertex  $s$  to a vertex  $v$ .
- To find the length of such a path.
- To find out if a graph contains cycles



## APPLICATION OF BFS: BIPARTITE GRAPH

? Graph  $G = (V, E)$  is **bipartite** iff  $V$  can be partitioned into two sets of nodes  $A$  and  $B$  such that each edge has one end in  $A$  and the other end in  $B$

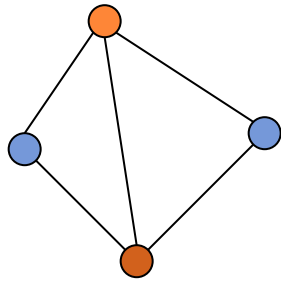
### Alternatively:

- Graph  $G = (V, E)$  is bipartite iff all its cycles have even length
- Graph  $G = (V, E)$  is bipartite iff nodes can be coloured using two colours

**Note:** graphs without cycles (trees) are bipartite

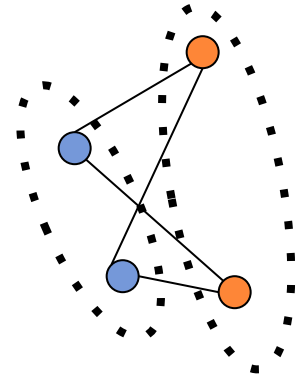
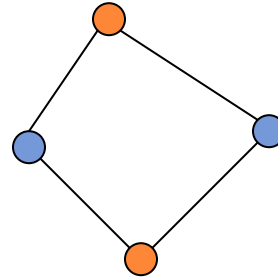


# APPLICATION OF BFS: BIPARTITE GRAPH



non bipartite

bipartite:



**Question:** given a graph  $G$ , how to test if the graph is bipartite?



# APPLICATION OF BFS: BIPARTITE GRAPH

```
For each vertex  $u$  in  $V[G] - \{s\}$ 
  do  $\text{color}[u] \leftarrow \text{WHITE}$ 
       $d[u] \leftarrow \infty$ 
       $\text{partition}[u] \leftarrow 0$ 
 $\text{color}[s] \leftarrow \text{GRAY}$ 
 $\text{partition}[s] \leftarrow 1$ 
 $d[s] \leftarrow 0$ 
 $Q \leftarrow [s]$ 
while Queue ' $Q$ ' is non-empty
  do  $u \leftarrow \text{head}[Q]$ 
      for each  $v$  in  $\text{Adj}[u]$  do
        if  $\text{partition}[u] = \text{partition}[v]$  then
          return 0
        else if  $\text{color}[v] \leftarrow \text{WHITE}$  then
           $\text{color}[v] \leftarrow \text{gray}$ 
           $d[v] = d[u] + 1$ 
           $\text{partition}[v] \leftarrow 3 - \text{partition}[u]$ 
          ENQUEUE ( $Q, v$ )
      DEQUEUE ( $Q$ )
   $\text{Color}[u] \leftarrow \text{BLACK}$ 
Return 1
```



# APPLICATION OF DFS:

## DETECTING CYCLE FOR DIRECTED GRAPH

**DFS\_visit( $u$ )**

$\text{color}(u) \leftarrow \text{GRAY}$

$d[u] \leftarrow \text{time} \leftarrow \text{time} + 1$

**for** each  $v$  adjacent to  $u$  **do**

**if**  $\text{color}[v] \leftarrow \text{GRAY}$  **then**

        return "cycle exists"

**else if**  $\text{color}[v] \leftarrow \text{WHITE}$  **then**

        predecessor[ $v$ ]  $\leftarrow u$

        DFS\_visit( $v$ )

$\text{color}[u] \leftarrow \text{BLACK}$

$f[u] \leftarrow \text{time} \leftarrow \text{time} + 1$



## APPLICATION OF DFS: DETECTING CYCLE FOR UNDIRECTED GRAPH

**DFS\_visit( $u$ )**

$\text{color}(u) \leftarrow \text{GRAY}$

$d[u] \leftarrow \text{time} \leftarrow \text{time} + 1$

**for** each  $v$  adjacent to  $u$  **do**

**if**  $\text{color}[v] \leftarrow \text{GRAY}$  and  $\pi[u] \neq v$  **then**

        return "cycle exists"

**else if**  $\text{color}[v] \leftarrow \text{WHITE}$  **then**

        predecessor[ $v$ ]  $\leftarrow u$

        DFS\_visit( $v$ )

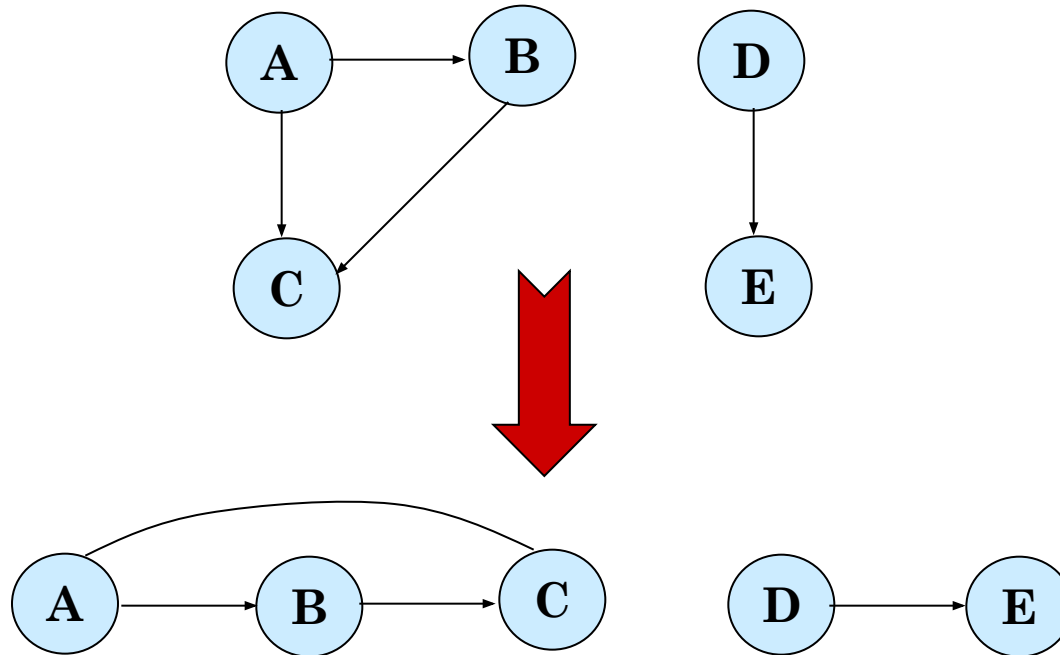
$\text{color}[u] \leftarrow \text{BLACK}$

$f[u] \leftarrow \text{time} \leftarrow \text{time} + 1$



# TOPOLOGICAL SORT

Want to “sort” a directed acyclic graph (DAG).



Think of original DAG as a **partial order**.

Want a **total order** that extends this partial order.



# TOPOLOGICAL SORT

- ? Performed on a **DAG**.
- ? Linear ordering of the vertices of  $G$  such that if  $(u, v) \in E$ , then  $u$  appears somewhere before  $v$ .

## Topological-Sort ( $G$ )

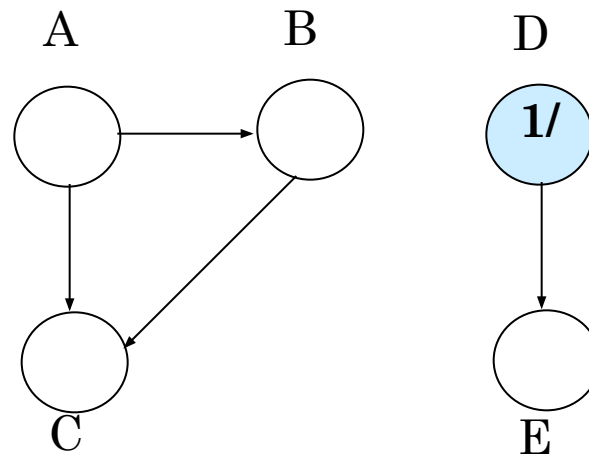
1. call DFS( $G$ ) to compute finishing times  $f[v]$  for all  $v \in V$
2. as each vertex is finished, insert it onto the front of a linked list
3. **return** the linked list of vertices

**Time:**  $\Theta(V + E)$ .





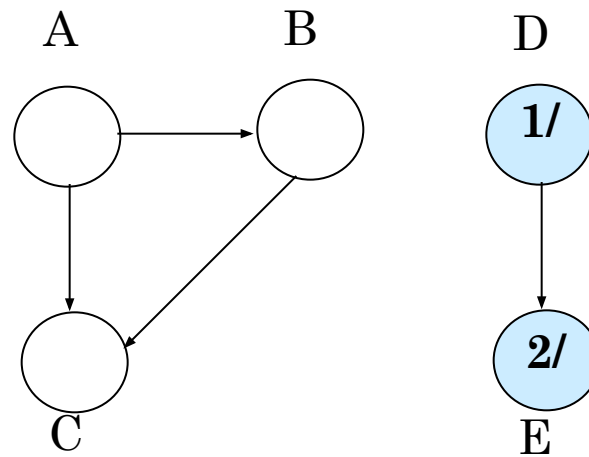
# EXAMPLE



**Linked List:**



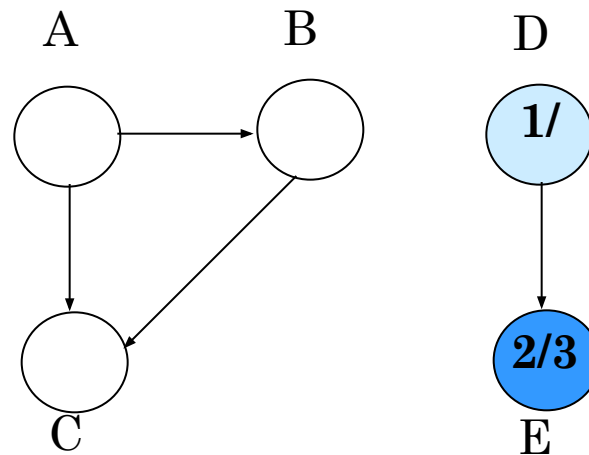
# EXAMPLE



**Linked List:**



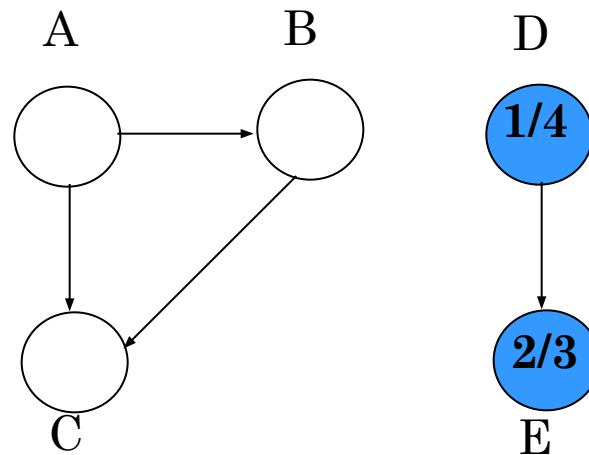
# EXAMPLE



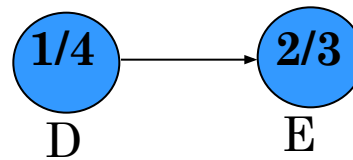
Linked List:



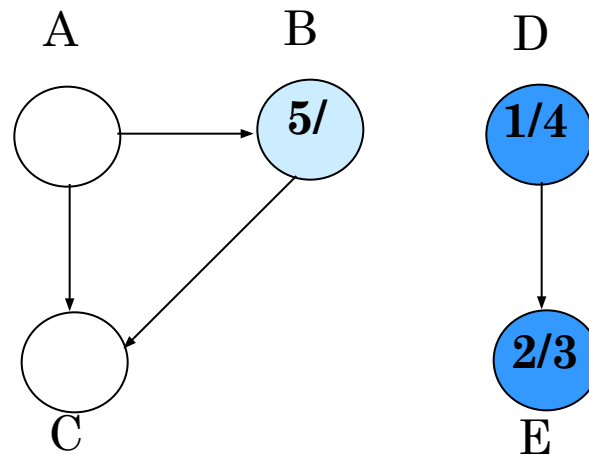
# EXAMPLE



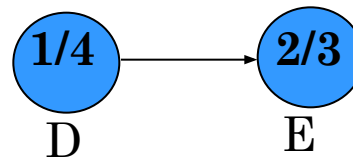
**Linked List:**



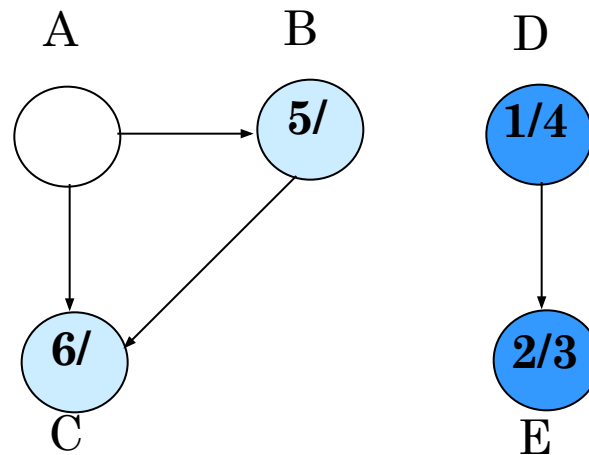
# EXAMPLE



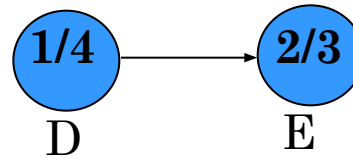
Linked List:



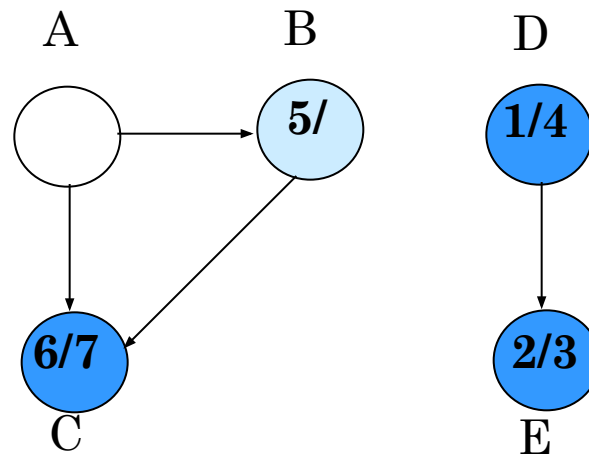
# EXAMPLE



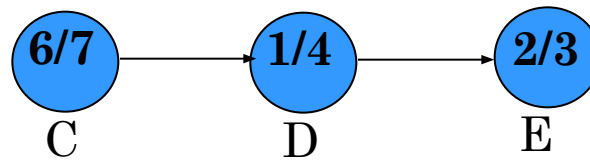
Linked List:



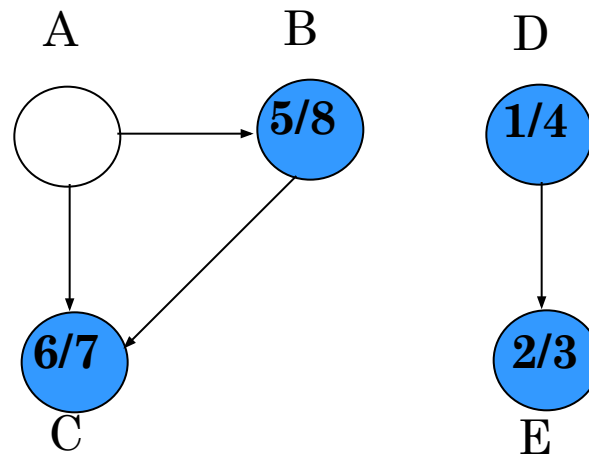
# EXAMPLE



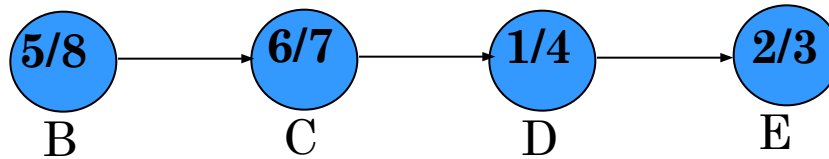
Linked List:



# EXAMPLE

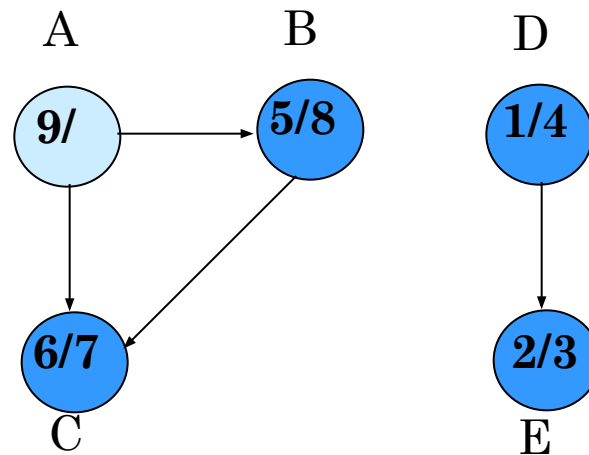


Linked List:

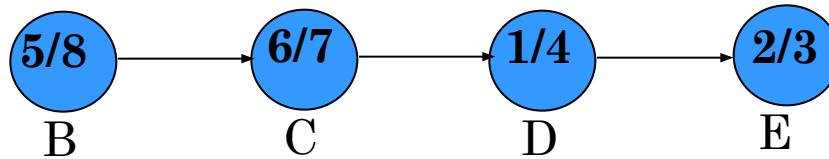




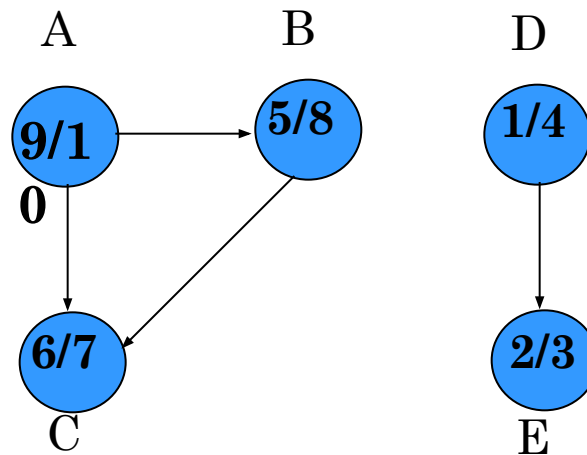
# EXAMPLE



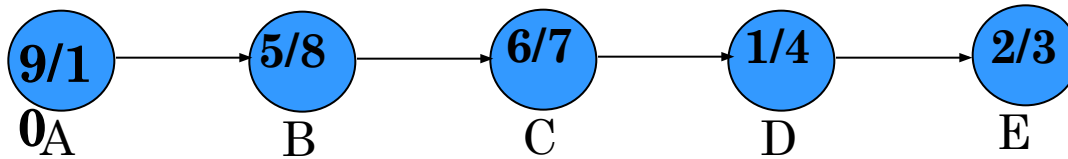
Linked List:

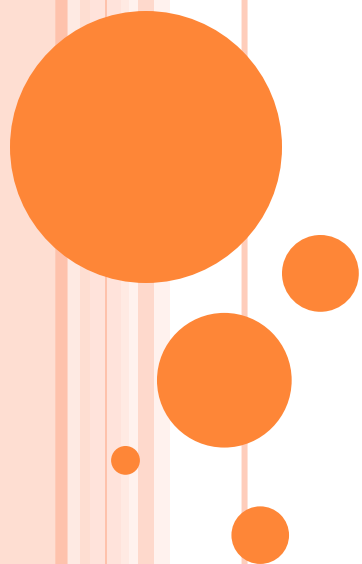


# EXAMPLE



Linked List:





**THE END**